

Technische Hochschule Ingolstadt
Fakultät Elektro- und Informationstechnik

Sensor Network Technologies and Sensor Fusion

Recap 4
Unscented Kalman Filter
Multi-Object Tracking
Particle Filters
SLAM, EKF SLAM, Fast-SLAM

June 29, 2026

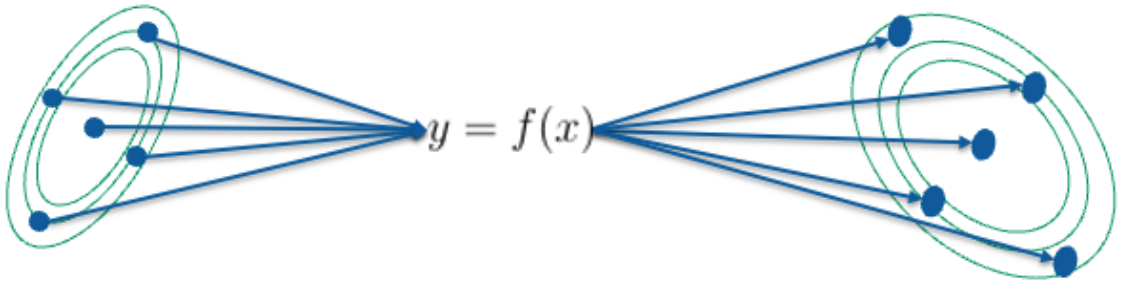
Welcome to the lecture!

Prof. Dr.-Ing. Björn Giesler
Technische Hochschule Ingolstadt
AIMotion, AI and Sensor Data Fusion
Email: bjoern.giesler@thi.de

- **Preparing for the Exam**
  - Use the slides!!!
    I am updating them to have a bit more consistent variable naming etc., but everything in the exam is in the slides, and in the exercises.
  - Use the exercises!
    I will not have time unfortunately to create more exercise sheets.
    Maybe AI can prepare some sheets for you?
  - I will publish a list of literature, Youtube videos etc. that can also help you prepare.
  - Study together, in pairs or groups!
    There's always someone in a group who has good questions, or good understanding.
- These recap slides are *not enough* to prepare for the exam. Use the original slides and the exercises.

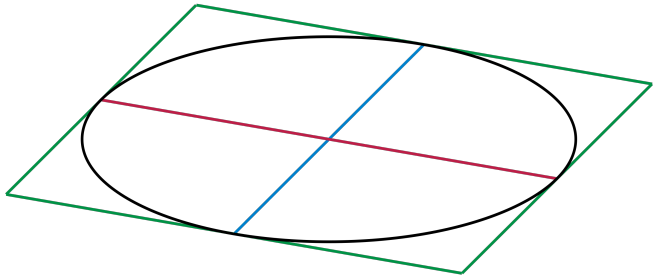
Unscented Kalman Filter

- **As we have seen, the Extended Kalman Filter helps us to get rid of *some* nonlinearities, provided they are reasonably well-behaved.**
- **It deteriorates for more complex functions.**
- **Enter the Unscented Kalman Filter. Basic idea:**
  - Sample the error distribution at *precisely chosen* points surrounding the mean
  - Transform these points through the state transition function
  - Use the transformed points to approximate the output distribution.

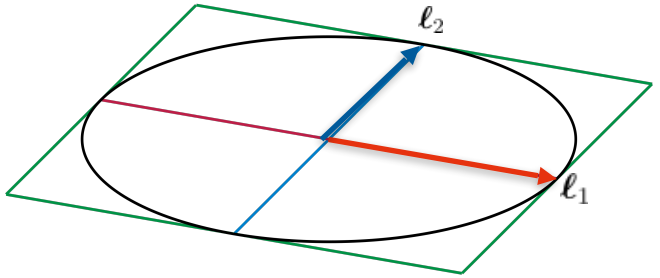


The Unscented Transform – Choosing Sigma Points

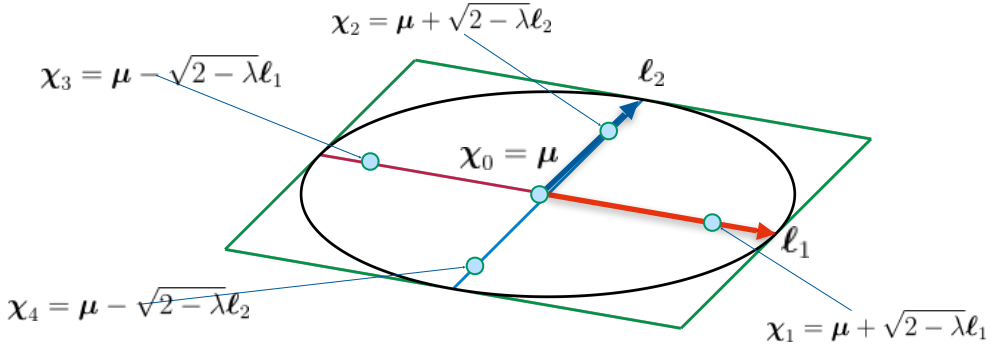
- **The Cholesky Decomposition is equivalent to a particular choice of conjugate axes of an ellipsoid.**
  - For an ellipsoid, two diameters are conjugate if and only if the tangent line to the ellipse at an endpoint of one diameter is parallel to the other diameter.
  - This is not the same as reconstructing the eigenvectors, but it gives symmetry axes that we can sample equidistant points on.



Two conjugate axes (red, blue) of an ellipsis $\sum$ ...



...obtained as column vectors ℓ_i of the Cholesky Decomposition $\Sigma = LL^\top$...



...give the positions of the sigma points.

The Unscented Transform – Choosing Sigma Points

- We generate $2n + 1$ sigma points

$$\boldsymbol{\chi}_0 = \boldsymbol{\mu},$$

$$\boldsymbol{\chi}_i = \begin{cases} \boldsymbol{\mu} + \sqrt{n+\lambda} \cdot \boldsymbol{\ell}_i & \text{if } i = 1, \ldots, n \\ \boldsymbol{\mu} - \sqrt{n+\lambda} \cdot \boldsymbol{\ell}_{i-n} & \text{if } i = n+1, \ldots, 2n \end{cases}$$

where ℓ_i is the i-th column of the Cholesky decomposition matrix $L \in R^{n \times n}$

$$\boldsymbol{L} \cdot \boldsymbol{L}^{\mathrm{T}} = \boldsymbol{P}_{k|k}$$

(L is a lower diagonal matrix) and λ is a chooseable parameter (a distance along the axis).

- Every sigma point $\boldsymbol{\chi}_i$ gets assigned a corresponding weight w_i with

$$w_0 = \frac{\lambda}{n+\lambda}$$

$$w_i = \frac{1}{2(n+\lambda)}, \quad i = 1 \ldots n$$

The weights are reconstructed from assumptions about the mean and variance ==> check original slides!

Unscented Kalman-Filter (UKF) – Prediction Step

1. Compute sigma-points $\chi_{i,t|t}$ based on the current estimate $\hat{x}_{t|t}$ and covariance $P_{t|t}$.
2. Predict: Predict the sigma-points employing the nonlinear dynamics, i.e.

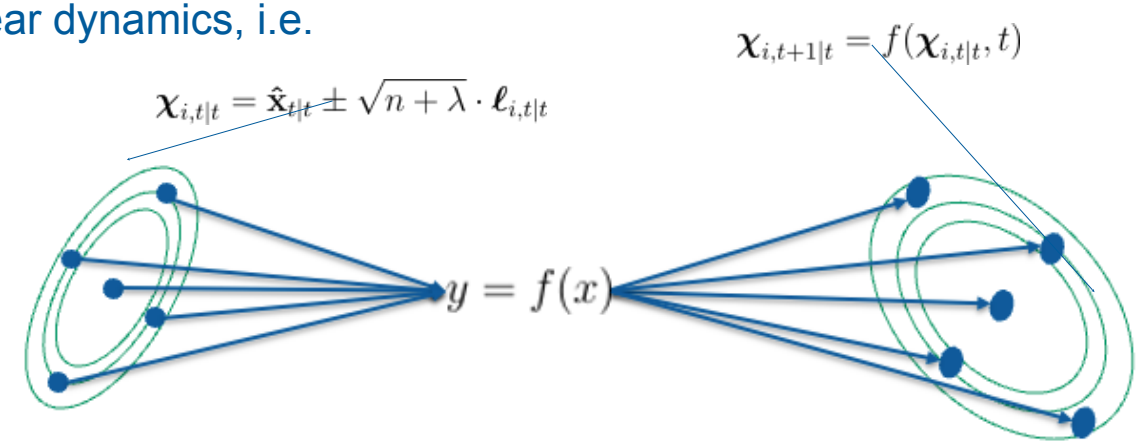
$$\chi_{i,t+1|t} = f(\chi_{i,t|t}, t)$$

3. Approximate the predicted mean

$$\hat{\mathbf{x}}_{t1+1|t} = \bar{\chi} = \sum_{i=0}^{2n} w_i \chi_{i,t+1|t}$$

and covariance:

$$P_{t+1|t} = Q_t + \sum_{i=0}^{2n} w_i (\chi_{i,t+1|t} - \bar{\chi})(\chi_{i,t+1|t} - \bar{\chi})^\top.$$



Unscented Kalman-Filter (UKF) – Measurement Update

1. Computing/Using sigma- points $\chi_{i,t+1|t}$ based on the predicted estimate $\hat{x}_{t+1|t}$ and covariance $P_{t+1|t}$
2. Compute a predicted measurement from the weighted sum of predictions for each sigma point:

$$\hat{\mathbf{z}}_{t+1|t} = \sum_{i=0}^{2n} w_i \mathbf{z}_{i,t+1|t} \qquad \text{with} \qquad \mathbf{z}_{i,t+1|t} = h(\boldsymbol{\chi}_{i,t+1|t}, t)$$

3. Approximate the measurement covariance and the cross-covariance between state and measurement:

$$P_{zz} = \sum_{i=0}^{2n} w_i (\mathbf{z}_{i,t+1|t} - \hat{\mathbf{z}}_{t+1|t})(\mathbf{z}_{i,t+1|t} - \hat{\mathbf{z}}_{t+1|t})^\top$$

$$P_{xz} = \sum_{i=0}^{2n} w_i (\boldsymbol{\chi}_{i,t+1|t} - \bar{\boldsymbol{\chi}})(\mathbf{z}_{i,t+1|t} - \hat{\mathbf{z}}_{t+1|t})^\top$$

4. Compute the Kalman-Gain and perform correction of state and covariance.

$$K_t = P_{xz} \cdot (R + P_{zz})^{-1}$$

$$\hat{\mathbf{x}}_{t+1|t+1} = \hat{\mathbf{x}}_{t+1|t} + K_t \cdot (\mathbf{z}_{t+1} - \hat{\mathbf{z}}_{t+1|t})$$

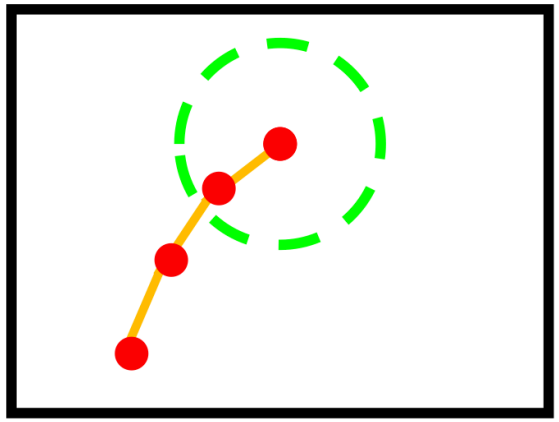
$$P_{t+1|t+1} = P_{t+1|t} - K_t P_{zz} K_t^\top.$$

Compare Kalman Filter and UKF Equations

	Linear	UKF
• Kalman Gain	$K_t = P_{t\|t-1} H^\top (H P_{t\|t-1} H^\top + R)^{-1}$	$K_t = P_{xz} \cdot (P_{zz} + R)^{-1}$
• Update Estimate	$\hat{\mathbf{x}}_{t+1\|t+1} = \mathbf{x}_{t+1\|t} + K_t \cdot (\mathbf{z}_{t+1} - H\hat{\mathbf{x}}_{t+1\|t})$	$\hat{\mathbf{x}}_{t+1\|t+} = \hat{\mathbf{x}}_{t+1\|t} + K_t \cdot (\mathbf{z}_{t+1} - \hat{\mathbf{z}}_{t+1\|t})$
• Update Covariance	$P_{t+1\|t+1} = P_{t+1\|t} - K_t H P_{t+1\|t}$	$P_{t+1\|t+1} = P_{t+1\|t} - K_t P_{zz} K_t^\top.$
• Predict	$\hat{\mathbf{x}}_{t+1\|t} = A\hat{\mathbf{x}}_{t\|t}$ $P_{t+1\|t} = A P_{t\|t} A^\top + Q$	$\hat{\mathbf{x}}_{t1+1\|t} = \bar{\boldsymbol{\chi}} = \sum_{i=0}^{2n} w_i \boldsymbol{\chi}_{i,t+1\|t}$ $P_{t+1\|t} = \sum_{i=0}^{2n} w_i (\boldsymbol{\chi}_{i,t+1\|t} - \bar{\boldsymbol{\chi}})(\boldsymbol{\chi}_{i,t+1\|t} - \bar{\boldsymbol{\chi}})^\top + Q$

Multi Object Tracking

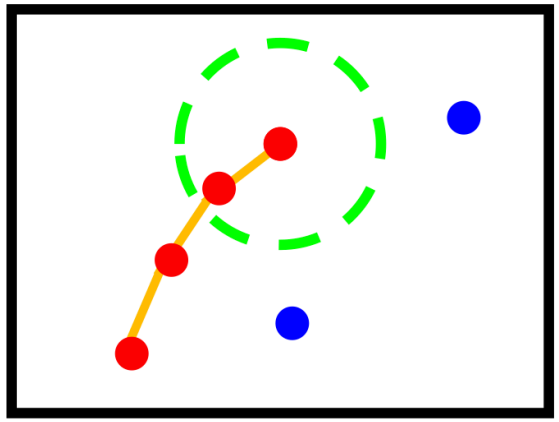
Motion Correspondence



1. Predictions may not be supported by measurements.

   Have the objects ceased to exist?
   ➔ remove them

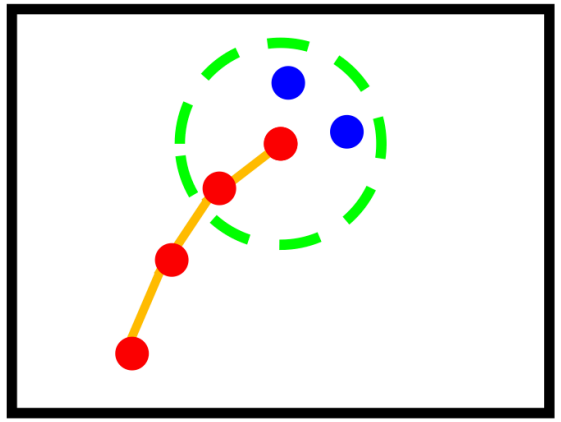
   Are they occluded?
   ➔ keep them



2. There may be unexpected measurements.

   Newly visible objects?
   ➔ set up tracks

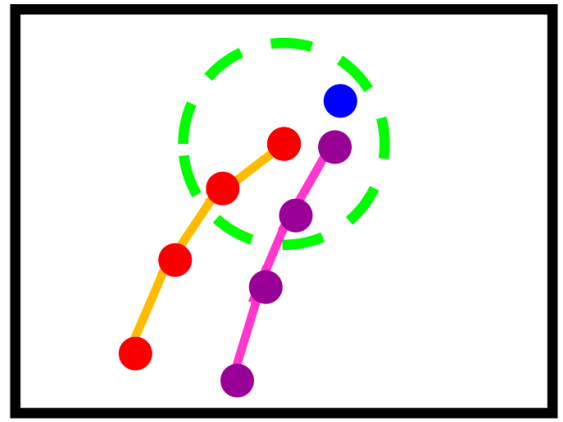
   Just noise?
   ➔ ignore them



3. More than one measurement may match a prediction.

   Which measurement is the correct one?

   What about the others?



4. A measurement may match to multiple predictions.

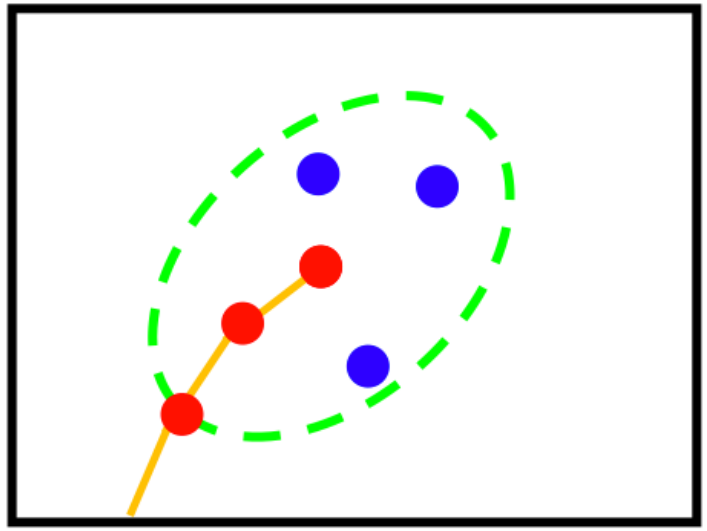
   Which track should the measurement be assigned to?

   What about the other track(s)?

Multi-Object Tracking Problem

- We represent a track by a state vector, e.g. $\mathbf{x} = \left[x, y, v_x, v_y\right]^\top$
- As the track evolves, we denote its state by the time index t: $\mathbf{x}^{(t)} = [x^{(t)}, y^{(t)}, v_x^{(t)}, v_y^{(t)}]^\top$
- At each time step, we get a set of observations / measurements: $\mathbf{Y}^{(t)} = \{\mathbf{y}_1^{(t)}, \ldots, \mathbf{y}_{M_t}^{(t)}\}$
- We now need to make the data association between tracks $\{\mathbf{x}_1^{(t)}, \ldots, \mathbf{x}_{N_t}^{(t)}\}$ and measurements $\{\mathbf{y}_1^{(t)}, \ldots, \mathbf{y}_{M_t}^{(t)}\}$:
- Association $z_l^{(t)} = j \text{ iff } \mathbf{y}_j^{(t)}$ is associated with $\mathbf{x}_l^{(t)}$.

The Mahalanobis Distance as a measure for track similarity



- Our Kalman Filter state of track $\mathbf{x}_l$ is given by the prediction and covariance $\hat{\mathbf{x}}_l^{(t)}, P_l'(t).$
- We have defined the *innovation* that measurement $\mathbf{z}_j$ brings to track $\mathbf{x}_l$ at time t as $\mathbf{i}_{j,l}^{(t)} = (\mathbf{z}_j^{(t)} - \hat{\mathbf{x}}_l^{(t)}).$
- With this, we can write the observation likelihood shortly as $p(\mathbf{z}_j^{(t)}|\mathbf{x}_l^{(t)} \sim \exp\{-\frac{1}{2}\mathbf{i}_{j,l}^{(t)\top} P_l'^{-1}\mathbf{i}_{j,l}^{\top}\}.$

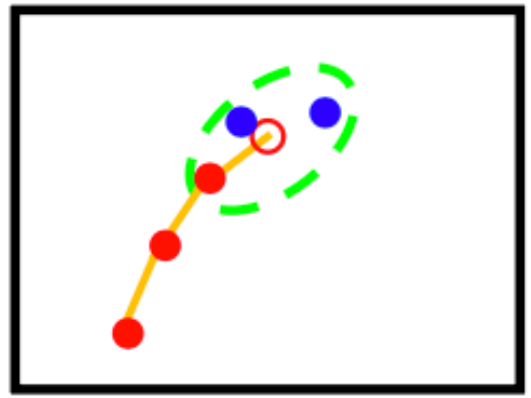
Mahalanobis Distance

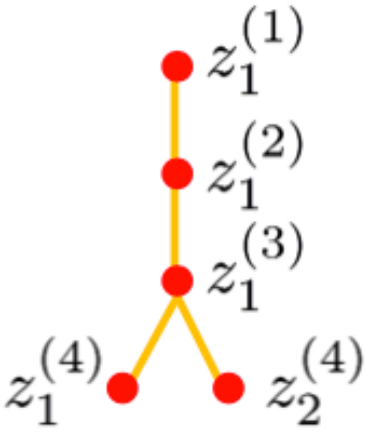
- We can define the ellipsoidal gating or validation volume as $V^{(t)}(\gamma) = \{\mathbf{z} : (\mathbf{y} - \hat{\mathbf{x}}_l^{(t)})^\top P_l'^{(t)-1}(\mathbf{y} - \hat{\mathbf{x}}_l^{(t)}) \leq \gamma\}.$
- This is equivalent to saying "all $\mathbf{y}$ with a Mahalanobis distance $\leq \gamma$".

$$z_l^{(t)} = \underset{j}{\operatorname{argmin}} \ (\hat{\mathbf{x}}_l^{(t)} - \mathbf{y}_j^{(t)})^\top P_l'^{-1}(\hat{\mathbf{x}}_l^{(t)} - \mathbf{y}_j^{(t)})$$

Track Splitting Filter

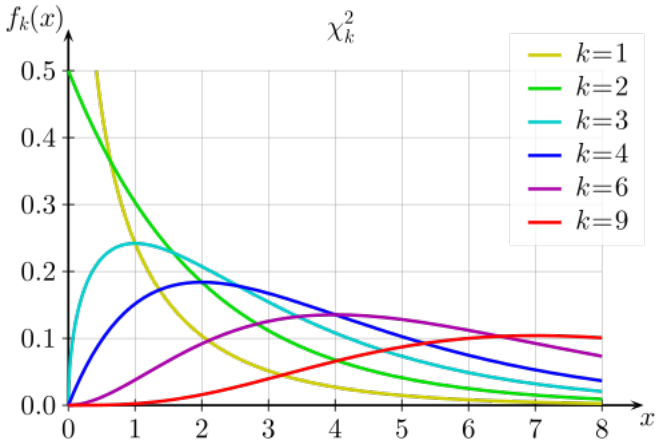
- **The Nearest-Neighbor filter still has uncertainties and options that can't really be resolved**
  - Uncertain decision making
  - ...through gating decision making functions
- **Rather than arbitrarily assigning the "closest" (by whatever norm) measurement, form a tree.**
- **Branches denote alternate assignments. No *decision* is made at this stage; decisions are postponed until additional measurements have been gathered.**
- **Potential problem:**
  Track trees can quickly become very large due to combinatorial explosion.
- **We need some measure of the likelihood so we can prune the tree.**





Track Splitting Filter – Pruning Strategies

- **In order to keep this feasible, need to apply pruning.**
- **Deleting unlikely tracks**
  - May be accomplished by comparing $\lambda(t)$, which follows a χ^2 distribution with $t \cdot n_z$ degrees of freedom, with a threshold α (set according to χ^2tables).
  - In long tracks, $\lambda(t)$ gets dominated by old terms and responds very slowly to new ones.
  - ➔ Use sliding window to limit track length, or implement exponential decay.
- **Merging track nodes**
  - If the state estimates of two track nodes are similar, merge them.
  - E.g. if both tracks validate identical subsequent measurments.
- **Rank tracks based on $\lambda(t)$, and only keep the most likely N tracks.**



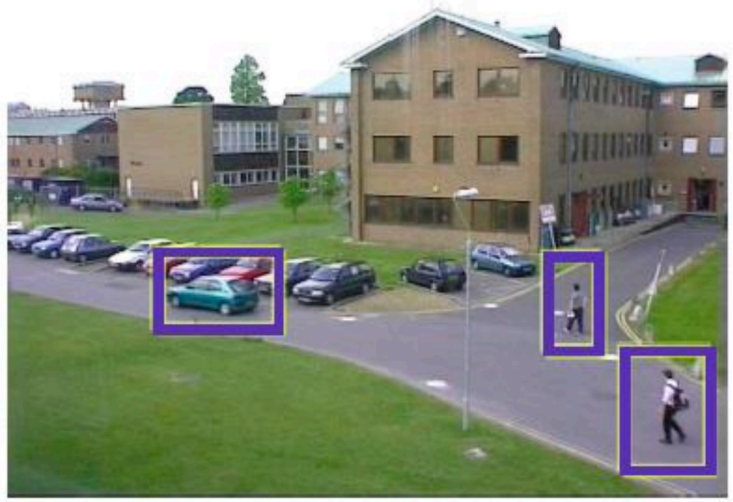
χ^2 distribution for different degrees of freedom

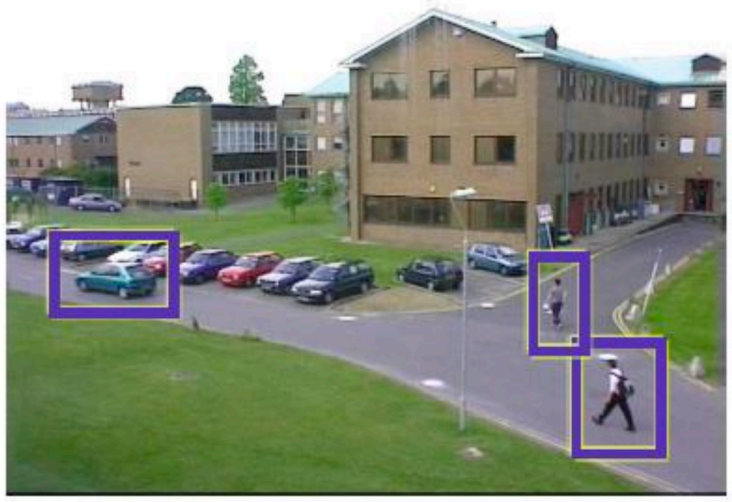
Data Association as Linear Assignment Problem

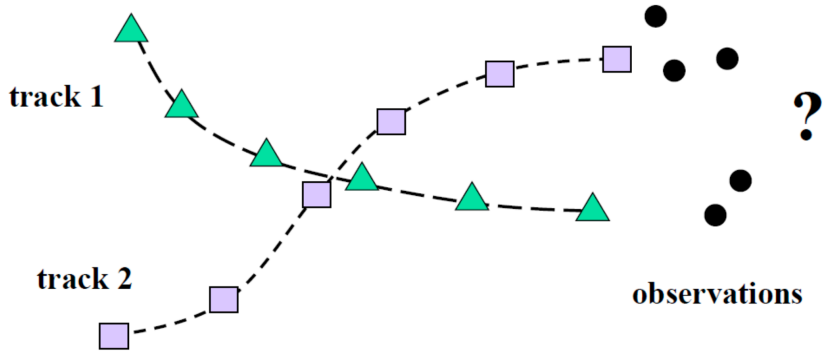
Slides credit: Robert Collins / Bastian Leibe

Back to Data Association.

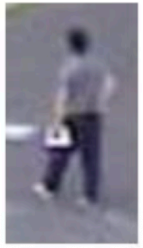
Goal: Match detections across frames.

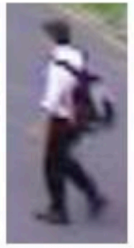


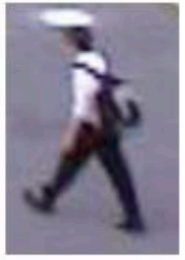




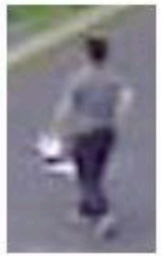












Linear Assignment Formulation

- **Form a matrix of pairwise similarity scores.**
- **Similarity could be based on**
  - Motion prediction (e.g. Mahalanobis Distance)
  - Visual appearance
  - Neural net output
  - …
- **Goal:**
  - Choose one match from each row and column to maximize the sum of scores

Frame $t+1$

Frame t			
	0.11	**0.95**	0.23
	0.85	0.25	**0.89**
	**0.90**	0.12	0.81

Formal Definition

- **Maximize**

$$\sum_{i=1}^{N}\sum_{j=1}^{M} w_{ij} z_{ij}$$

- **subject to**

$$\left.\begin{aligned} &\textstyle\sum_{j=1} z_{ij} = 1; i = 1, 2, \ldots, N \\ &\textstyle\sum_{i=1} z_{ij} = 1; j = 1, 2, \ldots, M \\ &z_{ij} \in \{0, 1\} \end{aligned}\right\}$$

These constraints ensure that Z is a permutation matrix.

- **The permutation matrix constraint ensures that we can only match up one object from each row and column.**
- **Note: Alternatively, we can minimize cost rather than maximizing weights --**

$$\arg\min_{z_{ij}} \sum_{i=1}^{N}\sum_{j=1}^{M} c_{ij} z_{ij}$$

Greedy Solution to Linear Assignment Problem

	1	2	3	4	5
1	0.95	0.76	0.62	0.41	0.06
2	0.23	0.46	0.79	0.94	0.35
3	0.61	0.02	0.92	0.92	0.81
4	0.49	0.82	0.74	0.41	0.01
5	0.89	0.44	0.18	0.89	0.14

- **Greedy Algorithm**
  - Find the largest score
  - Remove scores in same row and column from consideration
  - Repeat
- **Result: Sum of found scores**

Greedy Solution to Linear Assignment Problem

	1	2	3	4	5
1	0.95	0.76	0.62	0.41	0.06
2	0.23	0.46	0.79	0.94	0.35
3	0.61	0.02	0.92	0.92	0.81
4	0.49	0.82	0.74	0.41	0.01
5	0.89	0.44	0.18	0.89	0.14

- **Greedy Algorithm**
  - Find the largest score
  - Remove scores in same row and column from consideration
  - Repeat
- **Result:**

score = 0.95 + 0.94 + 0.92 + 0.82 + 0.14 = 3.77

Is this the best we can do?

Greedy Solution to Linear Assignment Problem

	1	2	3	4	5
1	0.95	0.76	0.62	0.41	0.06
2	0.23	0.46	0.79	0.94	0.35
3	0.61	0.02	0.92	0.92	0.81
4	0.49	0.82	0.74	0.41	0.01
5	0.89	0.44	0.18	0.89	0.14

Greedy Score: 3.77

	1	2	3	4	5
1	0.95	0.76	0.62	0.41	0.06
2	0.23	0.46	0.79	0.94	0.35
3	0.61	0.02	0.92	0.92	0.81
4	0.49	0.82	0.74	0.41	0.01
5	0.89	0.44	0.18	0.89	0.14

Optimal Score: 4.26

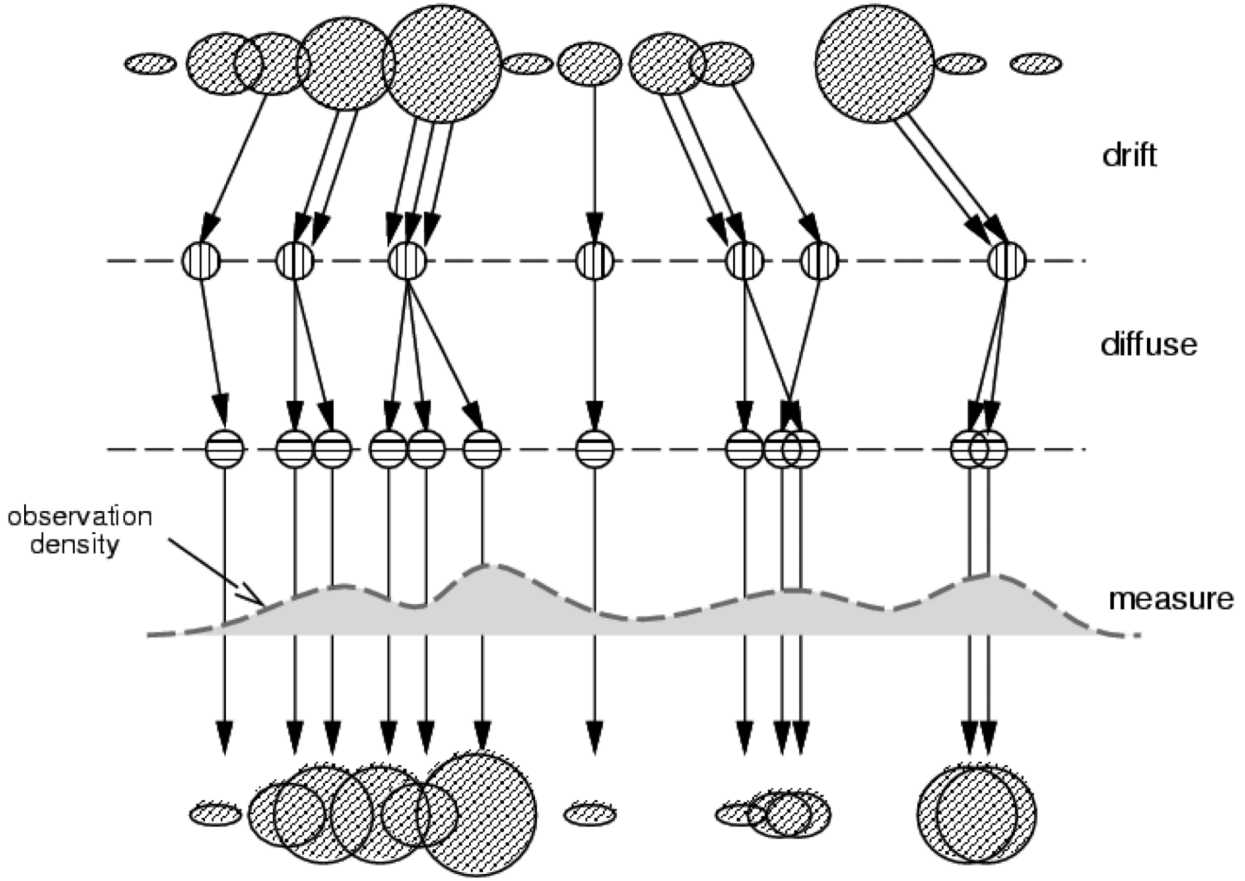
- **Greedy method is easy to program, quick to run, and yields "pretty good" results in practice.**
- **But often doesn't yield the optimal solution.**

Particle Filters



Beyond Gaussian Error Models

- We have seen through the Central Limit Theorem that given enough samples, each independent random variable is normal distributed.
- What if we can't *separate one from the other?*
- What if a single hypothesis is too limiting?

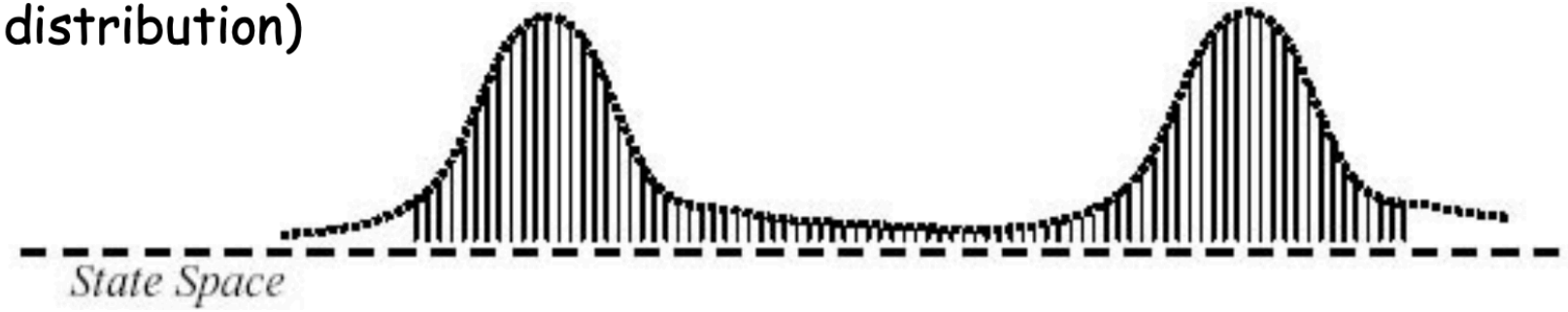


- Before the name "Particle Filters", the first stages were known as "Conditional Density Propagation", or "CONDENSATION".
- Fantastic section on Particle Filters can be found in Thrun's "Probabilistic Robotics", https://robots.stanford.edu/probabilistic-robotics/



1. Sample from prior belief $q(\mathbf{x})$ (e.g. the uniform distribution)

distribution)

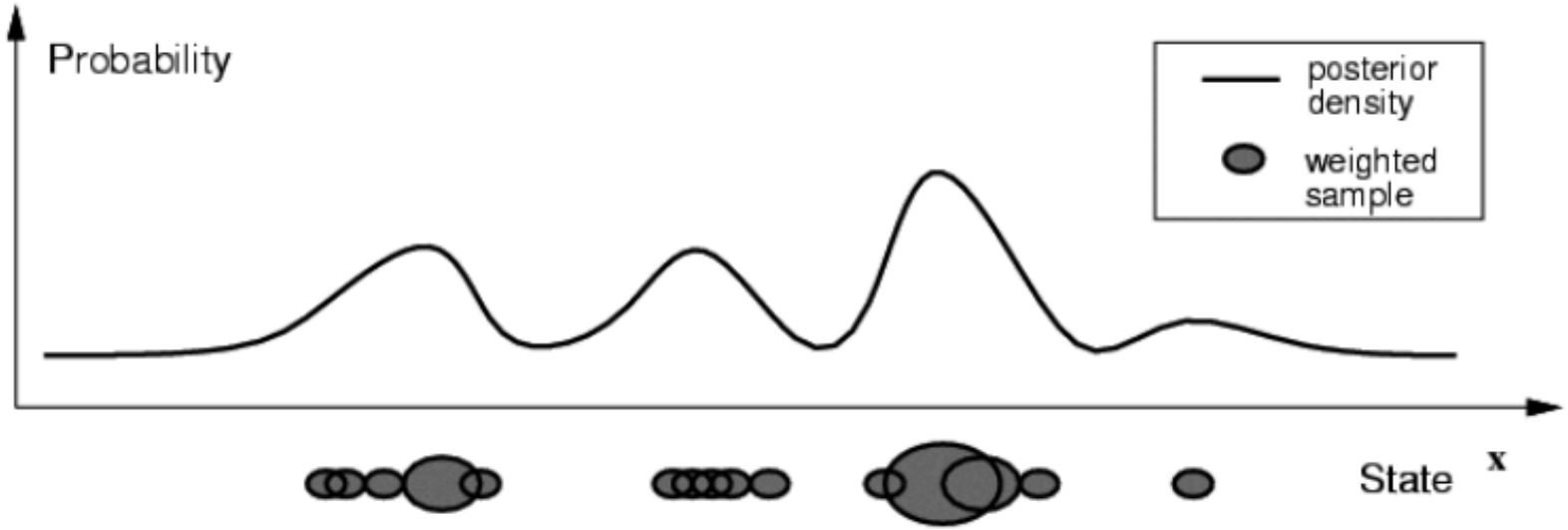


2. Compute importance weights, $w(\mathbf{x}) = p(\mathbf{x})/q(\mathbf{x})$



3. Resample particles according to importance weights to get $p(\mathbf{x})$.
Samples with high weights chosen many times, density reflects PDF.

Factored Sampling



- **Idea: Represent state distribution implicitly, without explicitly modelling its parameters.**
  - Prediction: Sample points from prior density for the state, $P(X)$.
  - Correction: Weigh the samples according to $P(Y|X)$.

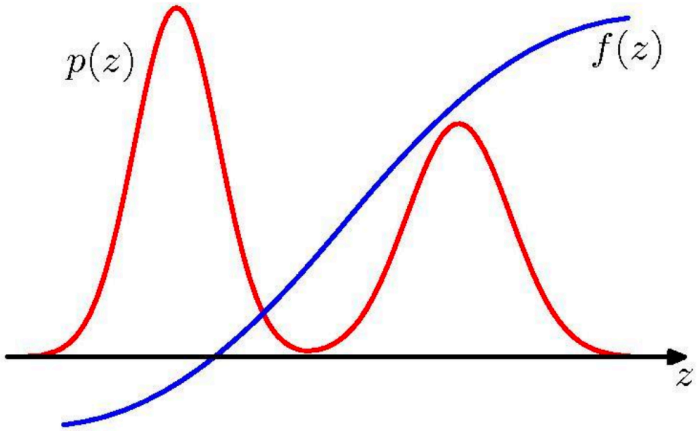
$$P(X_t|y_0,\ldots,y_t) = \frac{P(y_t|X_t)P(X_t|y_0,\ldots,y_{t-1})}{\int P(y_t|X_t)P(X_t|y_0,\ldots,y_{t-1})dX_t}$$

Monte Carlo Sampling

- **Objective:**
  - Evaluate *expectation* of a function $f(\mathbf{z})$

    regarding a probability distribution $p(\mathbf{z})$:

$$\mathbb{E}[f] = \int f(\mathbf{z})p(\mathbf{z})d\mathbf{z}.$$

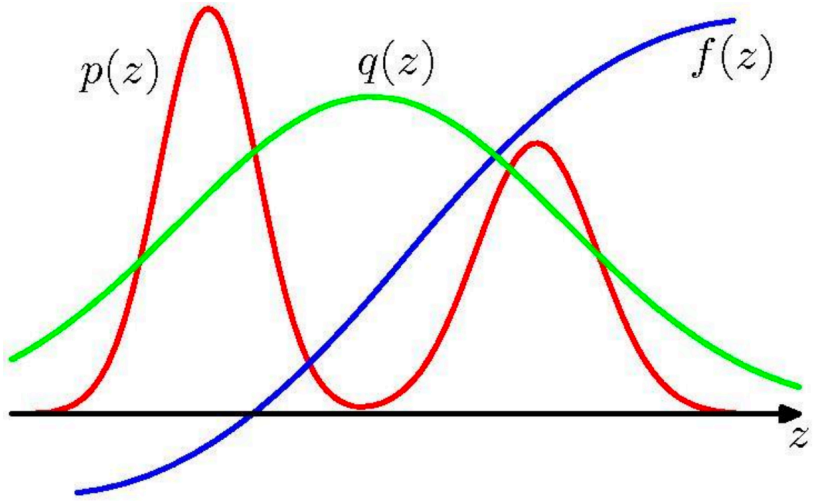


  - Monte Carlo Sampling idea:
    - Draw L independent samples $\mathbf{z}^{(l)}$ with $l = 1, \ldots, L$ from $p(\mathbf{z})$.
    - Approximate expectation with the finite sum $\hat{f} = \frac{1}{L}\sum_{l=1}^{L} f(\mathbf{z}^{(l)})$.
    - As long as the samples $\mathbf{z}^{(l)}$ do not depend on each other, but are independently drawn, $\mathbb{E}[\hat{f}] \approx \mathbb{E}[f]$.

    ➔ Unbiased estimate!

Importance Sampling

- **Let's consider an example** $F_N = \frac{1}{N} \sum_{i=1}^{N} \frac{f(\mathbf{x}_i)}{p(\mathbf{x}_i)}.$
  - f/p is the *importance weight* of a sample.
  - What can go wrong here?
  - If p is very small, f/p can get arbitrarily large!
    ➔ Design p such that f/p is bounded.
  - Effect: get more samples in "important" areas of f, i.e. where f is large.



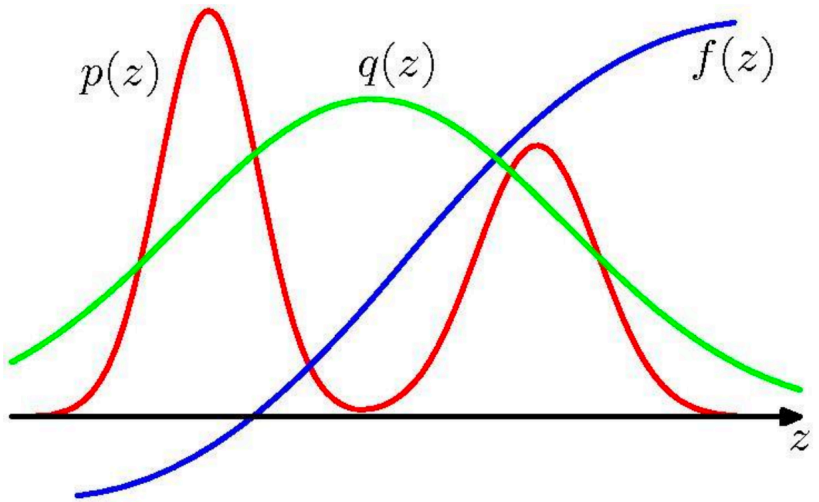
Background: Importance Sampling

- Idea: Use a proposal distribution $q(\mathbf{z})$ from which it is easy to draw samples and which is close in shape to f.
- Express expectations in the form of a finite sum over samples $\{\mathbf{z}^{(l)}\}$ drawn from $q(\mathbf{z})$.

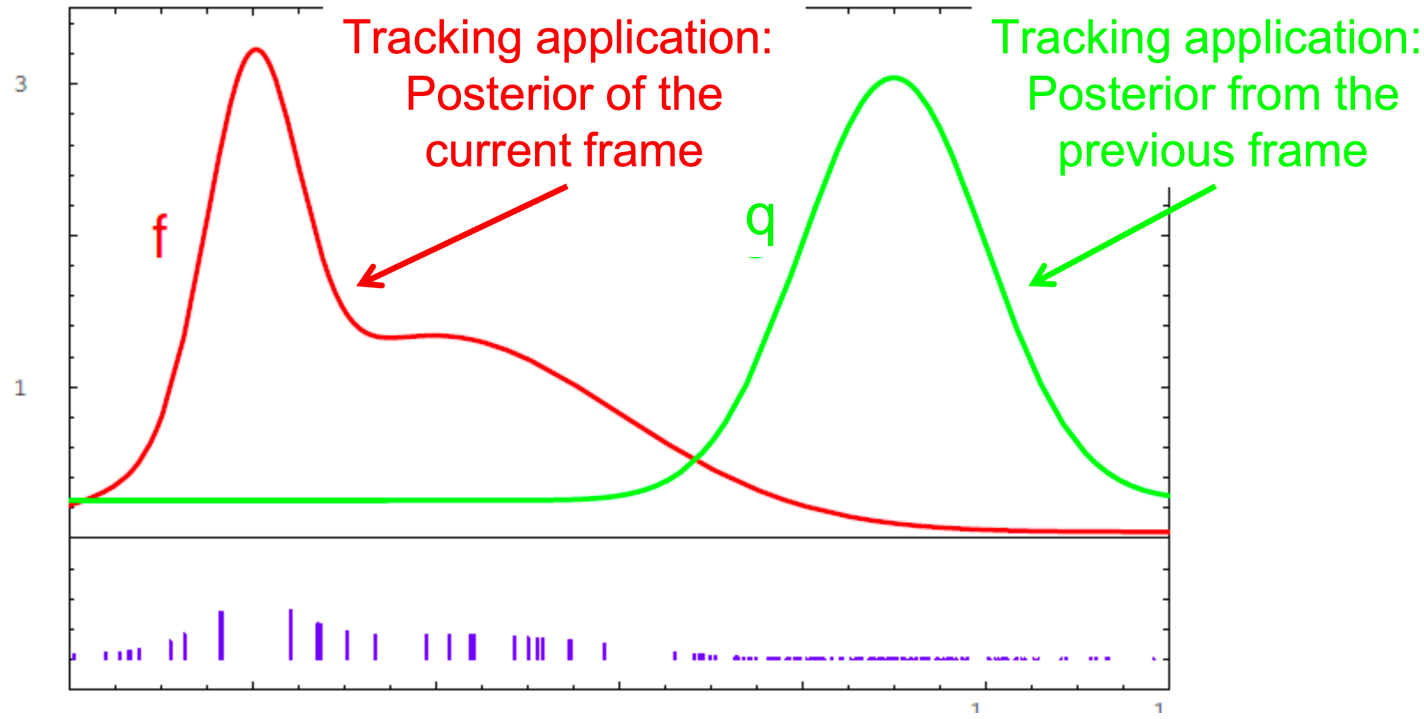
$$\begin{aligned}\mathbb{E}[f] &= \int f(\mathbf{z})p(\mathbf{z})d\mathbf{z} \\ &= \int f(\mathbf{z})\frac{p(\mathbf{z})}{q(\mathbf{z})}q(\mathbf{z})d\mathbf{z} \\ &\simeq \frac{1}{L}\sum_{l=1}^{L} f(\mathbf{z}^{(l)})\frac{p(\mathbf{z}^{(l)})}{q(\mathbf{z}^{(l)})}\end{aligned}$$

...with importance weights

$$r_l = \frac{p(\mathbf{z}^{(l)})}{q(\mathbf{z}^{(l)})}.$$



Tracking Application



- Goal: Approximate target density f.
- But instead of sampling from f directly, we can only sample from g.
- An idea of f is obtained by attaching the weight $f(\mathbf{x})/q(\mathbf{x})$ to each sample $\mathbf{x}$.

Importance Sampling for Bayesian Estimation

$$\begin{aligned}\mathbb{E}[f(X)] &= \int_X f(\mathbf{x}_{0:t}) p(\mathbf{x}_{0:t}|\mathbf{y}_{1:t}) d\mathbf{x}_{0:t} \\ &= \int_X f(\mathbf{x}_{0:t}) \frac{p(\mathbf{x}_{0:t}|\mathbf{y}_{1:t})}{q(\mathbf{x}_{0:t}|\mathbf{y}_{1:t})} q(\mathbf{x}_{0:t}|\mathbf{y}_{1:t}) d\mathbf{x}_{0:t}\end{aligned}$$

- **Applying Importance Sampling**
  - Characterize the posterior pdf using a set of samples ("particles") and their weights

$$\{\mathbf{x}_{0:t}^i, w_t^i\}_{i=1}^N$$

  - Then we can approximate the joint posterior by

$$p(\mathbf{x}_{0:t}|\mathbf{y}_{1:t}) \approx \sum_{i=1}^{N} w_t^i \delta(\mathbf{x}_{0:t} - \mathbf{x}_{0:t}^i).$$

Importance Sampling for Bayesian Estimation

$$\mathbb{E}[f(X)] = \int_X f(\mathbf{x}_{0:t}) p(\mathbf{x}_{0:t}|\mathbf{y}_{1:t}) d\mathbf{x}_{0:t}$$
$$= \int_X f(\mathbf{x}_{0:t}) \frac{p(\mathbf{x}_{0:t}|\mathbf{y}_{1:t})}{q(\mathbf{x}_{0:t}|\mathbf{y}_{1:t})} q(\mathbf{x}_{0:t}|\mathbf{y}_{1:t}) d\mathbf{x}_{0:t}$$

- **Applying Importance Sampling**
  - Draw the samples from the importance density $q(\mathbf{x}_{0:t}|\mathbf{y}_{1:t})$ with importance weights

$$w_t^i \propto \frac{p(\mathbf{x}_{0:t}|\mathbf{y}_{1:t})}{q(\mathbf{x}_{0:t}|\mathbf{y}_{1:t})}$$

  - Particle update:

$$\mathbf{x}_t \sim q(\mathbf{x}_t|\mathbf{x}_{t-1}^i, \mathbf{y}_t)$$

  - Weight update:

$$w_t^i = w_{t-1}^i \frac{p(\mathbf{y}_t|\mathbf{x}_t^i) p(\mathbf{x}_t^i|\mathbf{x}_{t-1}^i)}{q(\mathbf{x}|x_{t-1}^i, \mathbf{y}_t)}$$

The Degeneracy Phenomenon

- **Unavoidable problem with SIS.**
- **After a few iterations, most particles have negligible weights.**
- **Large computational effort for updating particles with very small contribution to** $p(\mathbf{x}_t|\mathbf{y}_{1:t})$.
- **Idea:**
  - Eliminate particles with low importance weights and increase the number of particles with high importance weight.
  - The *number of particles*, not their *weights*, represent the a posteriori density; so the weights are all set to uniform.

$$\{\mathbf{x}_t^i, w_t^i\}_{i=1}^N \rightarrow \left\{\mathbf{x}_t^{i*}, \frac{1}{N}\right\}_{i=1}^N.$$

  - The new set is generated by sampling with replacement from the discrete representation of $p(\mathbf{x}_t|\mathbf{y}_{1:t})$. The weights are used for multiplication of the particles:

$$P\{\mathbf{x}_t^{i*} = \mathbf{x}_t^j\} = w_t^j.$$

Resampling in Practice

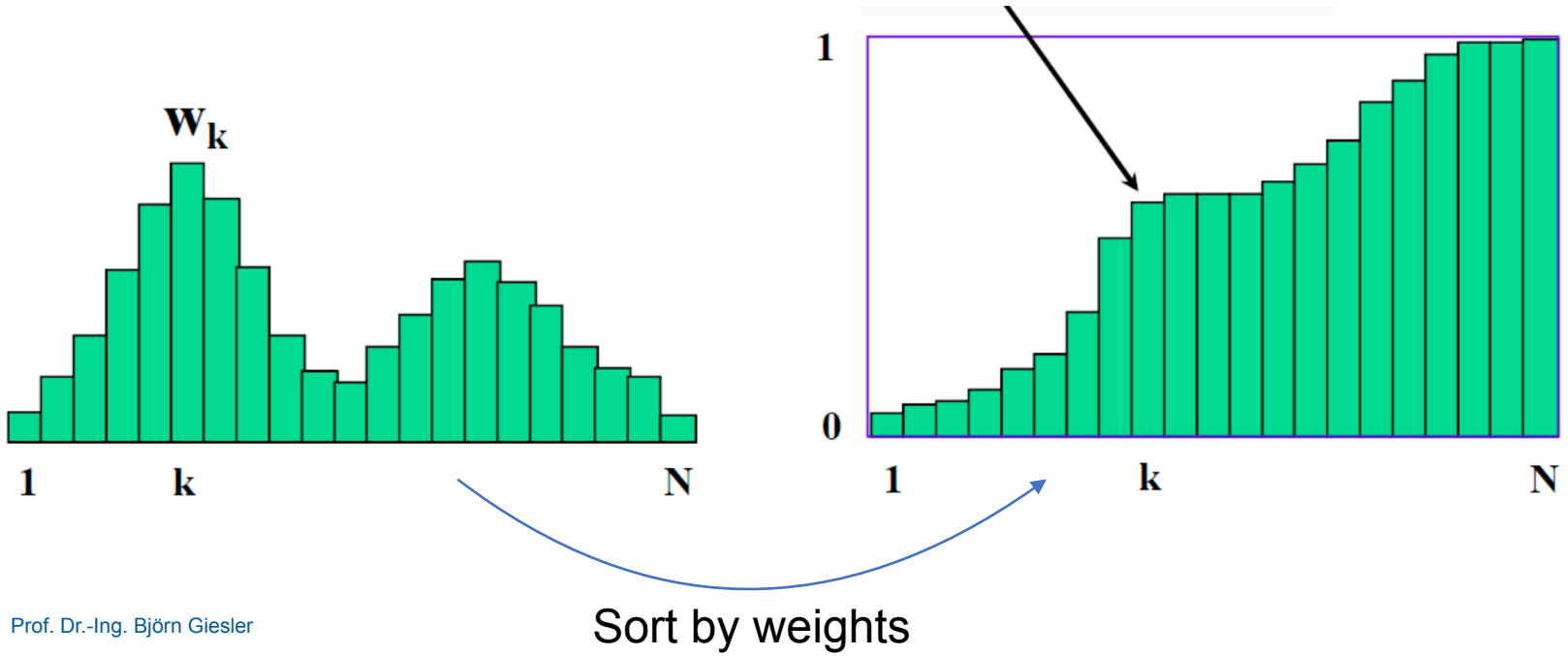
- We want to resample $\{\mathbf{x}_t^i\}_{i=1}^N$ from the discrete PDF given by the weighted samples $\{\mathbf{x}_t^i, w_t^i\}_{i=1}^N$.
- I.e. we want to draw N new samples $\{\mathbf{x}_t^i\}_{i=1}^N$ with replacement

  where the probability of drawing $\mathbf{x}_t^j$ is given by w_t^j.
- There are many algorithms for this – let's look at two simple ones.

Inverse Transform Sampling

- **Idea:**
  - It is easy to sample from a discrete distribution using the cumulative distribution function $F(x) = p(X \leq x)$.

$$c(k) = \frac{\sum_{i=1}^{k} w_i}{\sum_{i=1}^{N} w_i}$$

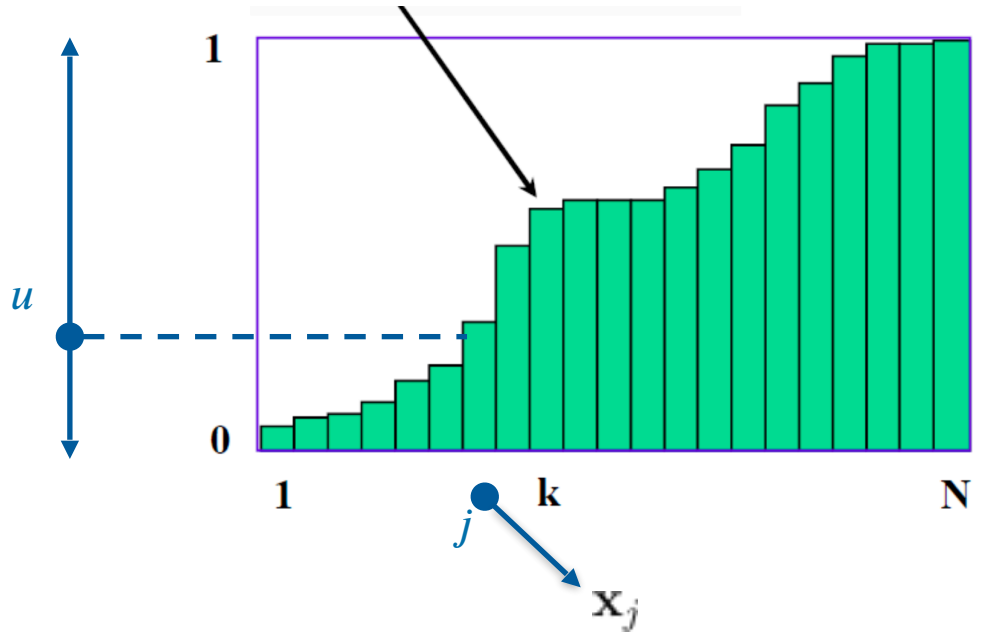


Inverse Transform Sampling

- **Idea:**
  - It is easy to sample from a discrete distribution using the cumulative distribution function $F(x) = p(X \leq x)$.

$$c(k) = \frac{\sum_{i=1}^{k} w_i}{\sum_{i=1}^{N} w_i}$$

- **Procedure:**
  1. Generate N samples for u in the range [0,1] according to uniform distribution.
  2. Visualize a horizontal line intersecting the bars.
  3. If index of intersected bar is j, output new sample $\mathbf{x}_j$.



SLAM

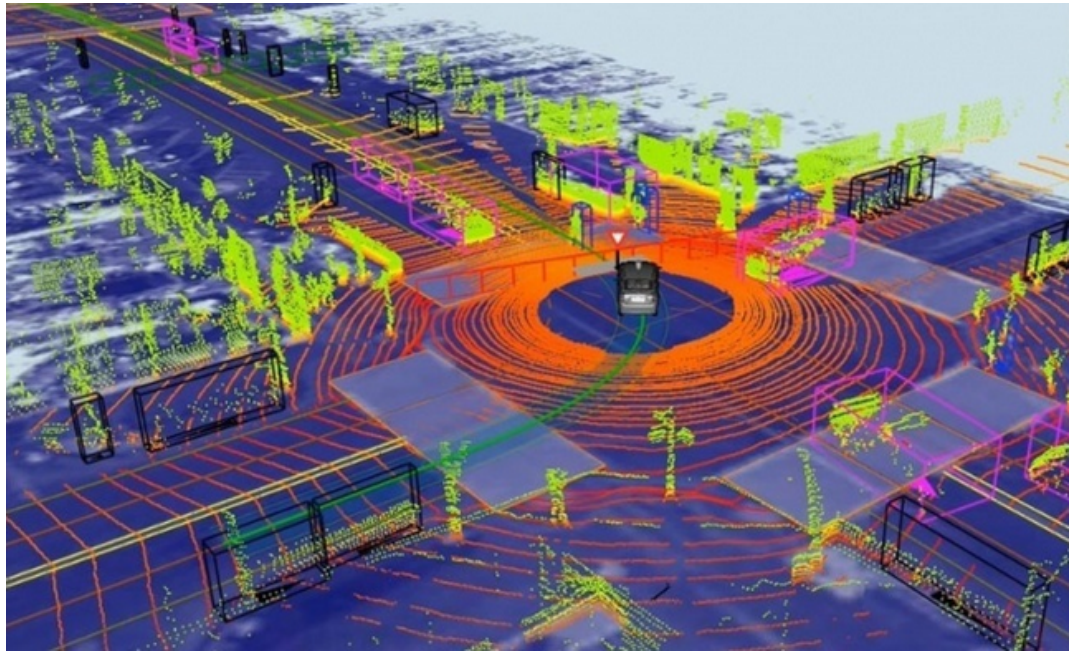
Simultaneous Localization and Mapping

Simultaneous Localization and Mapping (SLAM)

- Process of a robotic system to build a map and localize itself within this map at the same time
- Challenge: Hen-Egg Problem
  - Localizing itself requires a map
  - Mapping requires knowledge of the location
- Estimate robot state $\boldsymbol{x}_k$ and landmark $\boldsymbol{m} = \{\boldsymbol{m}_1, \ldots, \boldsymbol{m}_N\}$
- In probabilistic terms, the SLAM problem tries to derive the probability density

$$p(\boldsymbol{x}_k, \boldsymbol{m} | \boldsymbol{y}_{1:k}, \mathbf{u}_{1:k})$$

based on the history of measurements $\boldsymbol{y}_{1:k}$ and inputs $\boldsymbol{u}_{1:k}$

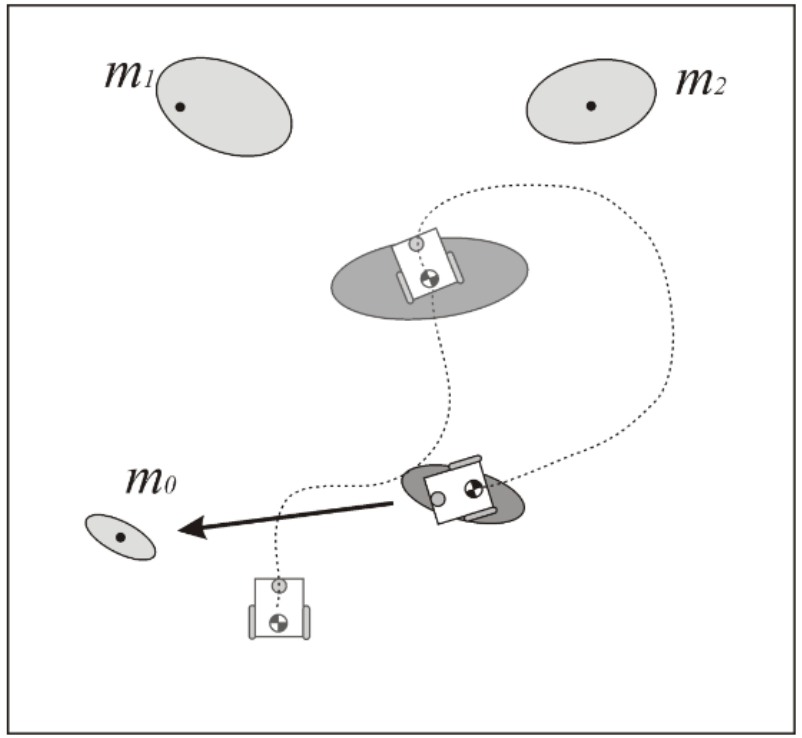


Lidar point cloud data

These slides owe a lot to Cyril Stachniss's SLAM course at Uni Freiburg

Simultaneous Localization and Mapping (SLAM)

- Initial position perfectly known, i. e. $\boldsymbol{P}_{xx} = \mathbf{0}$
- Landmark position $\boldsymbol{m}_0$ sensed with uncertainty
- When the robot moves, the uncertainty of its state estimate increases
- Now, two new landmarks are sensed with an uncertainty determined by the own position uncertainty and the measurement uncertainty
  - Landmark and state estimate become correlated
- Robot moves again, state estimate uncertainty increases
- Now the landmark m_0 is sensed again, which has a low uncertainty, this is referred to as loop-closure
  - Then the uncertainty of its state estimate is decreased
  - Simultaneously, all the uncertainties of the landmarks decrease

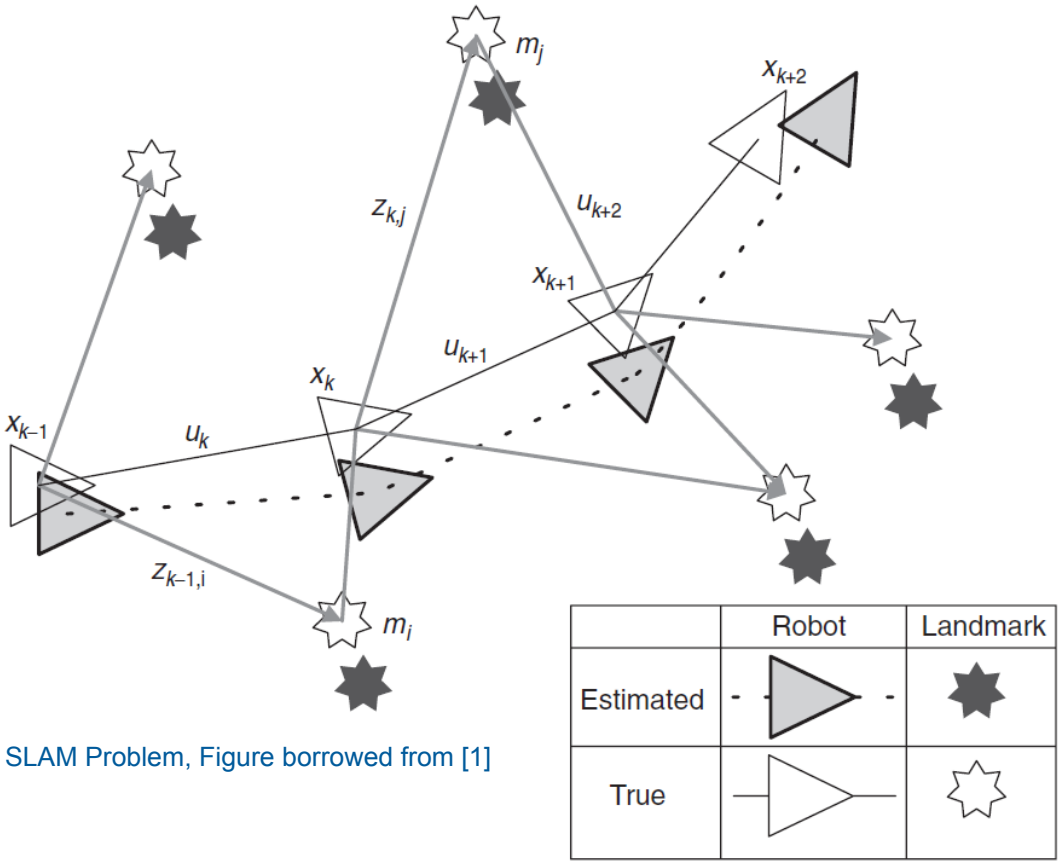


SLAM scenario, Figure borrowed from [2]

[2] https://www.cs.columbia.edu/~allen/F19/NOTES/slam_pka.pdf

Simultaneous Localization and Mapping (SLAM)

- Map built
  - based on noisy observations
  - taken from the uncertain position of the robot
- The landmarks can be considered as
  - Static
  - Stochastically independent in their error distribution
- However building up the map introduces correlation through the influence of the error in the position of the observed robot.
- Solution: Either
  - Model the stochastic correlation (EKF-SLAM) or
  - Cut apart the mapping and localization problems (FastSLAM).



SLAM Problem, Figure borrowed from [1]

[1] Durrant-White et. al.: Simultaneous Localization and Mapping: Part 1, IEEE Robotics & Automation Magazine, 2006

EKF-SLAM

EKF Application to SLAM

- Estimate robot's pose and locations of landmarks in the environment with one filter
- State space (for a robot moving in 2D) is

$$\mu_t = (\underbrace{x_t, y_t, \theta_t}_{\text{robot pose } \mathbf{x}_t}, \underbrace{m_{1,x}, m_{1,y}}_{\text{landmark } \mathbf{m}_1}, \ldots, \underbrace{m_{n,x}, m_{n,y}}_{\text{landmark } \mathbf{m}_n})^\top$$

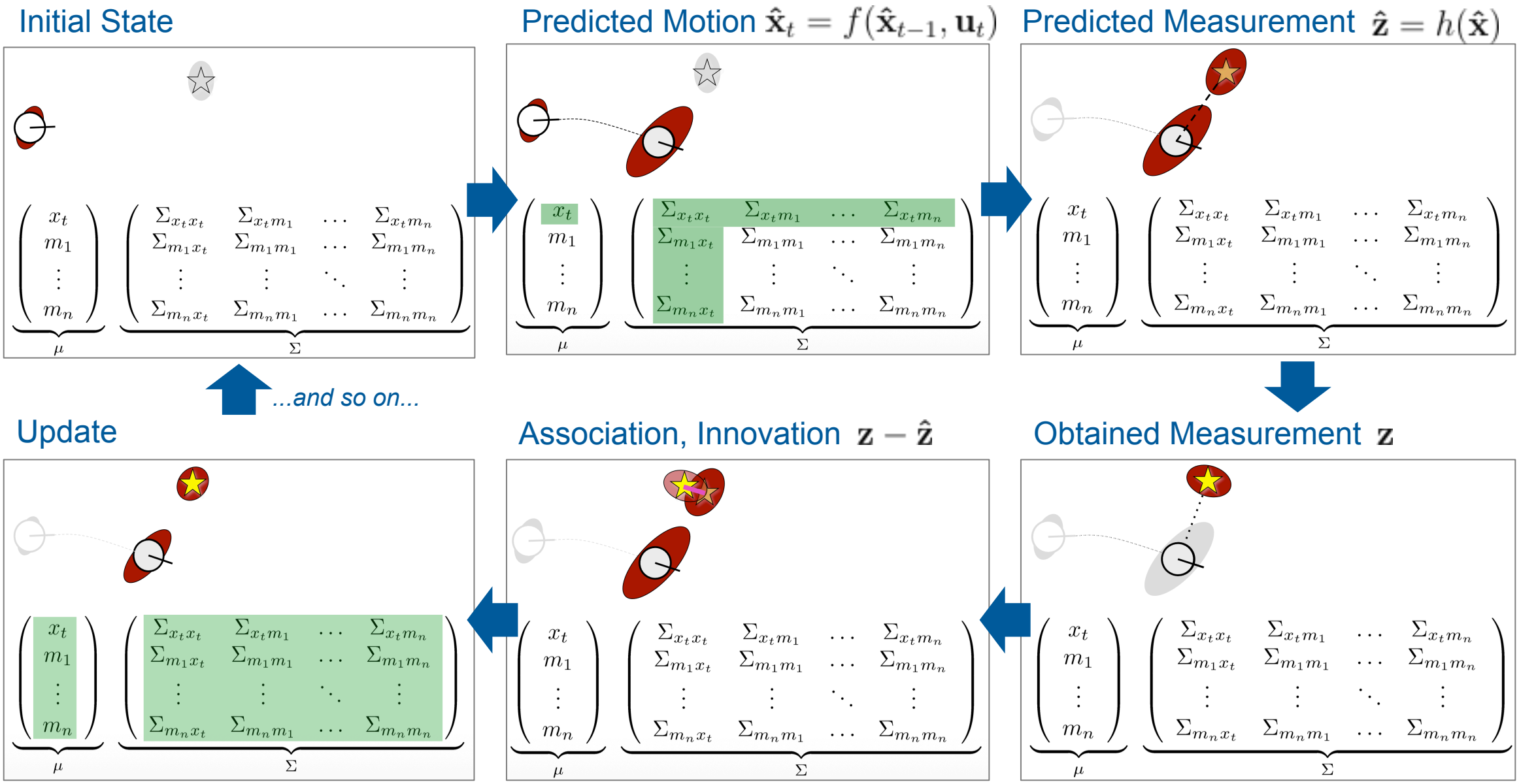
- Covariance matrix models joint uncertainties between robot pose and individual landmarks:

$$\Sigma = \begin{bmatrix} \Sigma_{\mathbf{x}_t\mathbf{x}_t} & \Sigma_{\mathbf{x}_t\mathbf{m}_1} & \cdots & \Sigma_{\mathbf{x}_t\mathbf{m}_n} \\ \Sigma_{\mathbf{m}_1\mathbf{x}_t} & \Sigma_{\mathbf{m}_1\mathbf{m}_1} & \cdots & \Sigma_{\mathbf{m}_1\mathbf{m}_n} \\ \vdots & \vdots & \ddots & \vdots \\ \Sigma_{\mathbf{m}_n\mathbf{x}_t} & \Sigma_{\mathbf{m}_n\mathbf{m}_1} & \cdots & \Sigma_{\mathbf{m}_n\mathbf{m}_n} \end{bmatrix}$$

or more compact:

$$\mathbf{s} = \begin{bmatrix} \mathbf{x} \\ \mathbf{m} \end{bmatrix}$$

$$\Sigma = \begin{bmatrix} \Sigma_{\mathbf{xx}} & \Sigma_{\mathbf{xm}} \\ \Sigma_{\mathbf{mx}} & \Sigma_{\mathbf{mm}} \end{bmatrix}$$


Initial State
Predicted Motion $\hat{\mathbf{x}}_t = f(\hat{\mathbf{x}}_{t-1}, \mathbf{u}_t)$
Predicted Measurement $\hat{\mathbf{z}} = h(\hat{\mathbf{x}})$
$\begin{pmatrix} x_t \\ m_1 \\ \vdots \\ m_n \end{pmatrix}$ μ
$\begin{pmatrix} \Sigma_{x_t x_t} & \Sigma_{x_t m_1} & \cdots & \Sigma_{x_t m_n} \\ \Sigma_{m_1 x_t} & \Sigma_{m_1 m_1} & \cdots & \Sigma_{m_1 m_n} \\ \vdots & \vdots & \ddots & \vdots \\ \Sigma_{m_n x_t} & \Sigma_{m_n m_1} & \cdots & \Sigma_{m_n m_n} \end{pmatrix}$ Σ
...and so on...
Update
Association, Innovation $\mathbf{z} - \hat{\mathbf{z}}$
Obtained Measurement $\mathbf{z}$

EKF-SLAM: Concrete Example

Setup

- Mobile robot that moves in the 2D plane

$$\mathbf{x} = (x, y, \theta)^\top$$

- Velocity-based motion model as input
  - Forward velocity v
  - Turn rate ω

$$\mathbf{u} = (v, \omega)^\top$$

- Observation of point landmarks
- Sensor delivering range and azimuth (bearing)
- Known data association
- Known number of landmarks

EKF Algorithm:

$$\hat{\mathbf{x}}_{t+1|t} = f(\hat{\mathbf{x}}_{t|t}, \mathbf{u}_t)$$

$$P_{t+1|t} = F P_{t|t} F^\top + Q_t$$

$$K_{t+1} = P_{t+1|t} H_{t+1}^\top (H_{t+1} P_{t+1|t} H_{t+1}^\top + R_{t+1})^{-1}$$

$$\hat{\mathbf{x}}_{t+1|t+1} = \hat{\mathbf{x}}_{t+1|t} + K_{t+1}(\mathbf{z}_{t+1} - h(\hat{\mathbf{x}}_{t+1|t}))$$

$$P_{t+1|t+1} = (I - K_{t+1} H_{t+1}) P_{t+1|t}$$

Prediction Step: Robot Motion

- Motion in the plane is

$$\begin{bmatrix} x' \\ y' \\ \theta' \end{bmatrix} = \underbrace{\begin{bmatrix} x \\ y \\ \theta \end{bmatrix} + \begin{bmatrix} -\frac{v_t}{\omega_t}\sin\theta + \frac{v_t}{\omega_t}\sin(\theta+\omega_t\Delta t) \\ \frac{v_t}{\omega_t}\cos\theta - \frac{v_t}{\omega_t}\cos(\theta+\omega_t\Delta t) \\ \omega_t\Delta t \end{bmatrix}}_{f((x,y,\theta)^\top,\mathbf{u}_t)}$$

- We map this to the 2N+3-dimensional space with

$$\begin{bmatrix} x' \\ y' \\ \theta' \\ \vdots \end{bmatrix} = \underbrace{\begin{bmatrix} x \\ y \\ \theta \\ \vdots \end{bmatrix} + \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \\ \vdots & \vdots & \vdots \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} -\frac{v_t}{\omega_t}\sin\theta + \frac{v_t}{\omega_t}\sin(\theta+\omega_t\Delta t) \\ \frac{v_t}{\omega_t}\cos\theta - \frac{v_t}{\omega_t}\cos(\theta+\omega_t\Delta t) \\ \omega_t\Delta t \end{bmatrix}}_{f(\mu_t,\mathbf{u}_t)}$$

$$\hat{\mathbf{x}}_{t+1|t} = f(\hat{\mathbf{x}}_{t|t}, \mathbf{u}_t)$$

$$P_{t+1|t} = FP_{t|t}F^\top + Q_t$$

$$K_{t+1} = P_{t+1|t}H_{t+1}^\top(H_{t+1}P_{t+1|t}H_{t+1}^\top + R_{t+1})^{-1}$$

$$\hat{\mathbf{x}}_{t+1|t+1} = \hat{\mathbf{x}}_{t+1|t} + K_{t+1}(\mathbf{z}_{t+1} - h(\hat{\mathbf{x}}_{t+1|t}))$$

$$P_{t+1|t+1} = (I - K_{t+1}H_{t+1})P_{t+1|t}$$

As we can see, $f(\bullet)$ only affects the motion and not the landmarks.

Deriving the Jacobian of the motion

$$
\begin{aligned}
F_t^x &= \frac{\partial}{\partial (x,y,\theta)^\top} \left[\begin{pmatrix} x \\ y \\ \theta \end{pmatrix} + \begin{pmatrix} -\frac{v_t}{\omega_t}\sin\theta + \frac{v_t}{\omega_t}\sin(\theta+\omega_t\Delta t) \\ \frac{v_t}{\omega_t}\cos\theta - \frac{v_t}{\omega_t}\cos(\theta+\omega_t\Delta t) \\ \omega_t \Delta t \end{pmatrix} \right] \\
&= I + \frac{\partial}{\partial (x,y,\theta)^\top} \begin{pmatrix} -\frac{v_t}{\omega_t}\sin\theta + \frac{v_t}{\omega_t}\sin(\theta+\omega_t\Delta t) \\ \frac{v_t}{\omega_t}\cos\theta - \frac{v_t}{\omega_t}\cos(\theta+\omega_t\Delta t) \\ \omega_t \Delta t \end{pmatrix} \\
&= I + \begin{pmatrix} 0 & 0 & -\frac{v_t}{\omega_t}\cos\theta + \frac{v_t}{\omega_t}\cos(\theta+\omega_t\Delta t) \\ 0 & 0 & -\frac{v_t}{\omega_t}\sin\theta + \frac{v_t}{\omega_t}\sin(\theta+\omega_t\Delta t) \\ 0 & 0 & 0 \end{pmatrix} \\
&= \begin{pmatrix} 1 & 0 & \frac{v_t}{\omega_t}(\cos(\theta+\omega_t\Delta t) - \cos\theta) \\ 0 & 1 & \frac{v_t}{\omega_t}(\sin(\theta+\omega_t\Delta t) - \sin\theta) \\ 0 & 0 & 1 \end{pmatrix}
\end{aligned}
$$

$$\hat{\mathbf{x}}_{t+1|t} = f(\hat{\mathbf{x}}_{t|t}, \mathbf{u}_t)$$

$$P_{t+1|t} = FP_{t|t}F^\top + Q_t$$

$$K_{t+1} = P_{t+1|t}H_{t+1}^\top (H_{t+1}P_{t+1|t}H_{t+1}^\top + R_{t+1})^{-1}$$

$$\hat{\mathbf{x}}_{t+1|t+1} = \hat{\mathbf{x}}_{t+1|t} + K_{t+1}(\mathbf{z}_{t+1} - h(\hat{\mathbf{x}}_{t+1|t}))$$

$$P_{t+1|t+1} = (I - K_{t+1}H_{t+1})P_{t+1|t}$$

Prediction Equations Complete!

$$\Sigma_{t+1|t} = F\Sigma_{t|t}F^\top + Q$$
$$= \begin{pmatrix} F_t^x & 0 \\ 0 & I \end{pmatrix} \begin{pmatrix} \Sigma_{xx} & \Sigma_{xm} \\ \Sigma_{mx} & \Sigma_{mm} \end{pmatrix} \begin{pmatrix} (F_t^x)^\top & 0 \\ 0 & I \end{pmatrix} + Q$$
$$= \begin{pmatrix} F_t^x \Sigma xx (F_t^x)^\top & F_t^x \Sigma_{xm} \\ (F_t^x \Sigma_{xm})^\top & \Sigma_{mm} \end{pmatrix} + Q$$

$$\hat{\mathbf{x}}_{t+1|t} = f(\hat{\mathbf{x}}_{t|t}, \mathbf{u}_t)$$

$$P_{t+1|t} = FP_{t|t}F^\top + Q_t$$

$$K_{t+1} = P_{t+1|t}H_{t+1}^\top (H_{t+1}P_{t+1|t}H_{t+1}^\top + R_{t+1})^{-1}$$

$$\hat{\mathbf{x}}_{t+1|t+1} = \hat{\mathbf{x}}_{t+1|t} + K_{t+1}(\mathbf{z}_{t+1} - h(\hat{\mathbf{x}}_{t+1|t}))$$

$$P_{t+1|t+1} = (I - K_{t+1}H_{t+1})P_{t+1|t}$$

EKF-SLAM: Correction Step

- We assume a known (and correct ☺) data association.
- i-th measurement at time t observes the landmark with index j:

$$c_t^i = j$$

- If the landmark was unobserved previously, initialize it
- Compute the expected observation
- Compute the Jacobian of h
- Compute the Kalman gain

$$\hat{\mathbf{x}}_{t+1|t} = f(\hat{\mathbf{x}}_{t|t}, \mathbf{u}_t)$$

$$P_{t+1|t} = FP_{t|t}F^\top + Q_t$$

$$K_{t+1} = P_{t+1|t}H_{t+1}^\top(H_{t+1}P_{t+1|t}H_{t+1}^\top + R_{t+1})^{-1}$$

$$\hat{\mathbf{x}}_{t+1|t+1} = \hat{\mathbf{x}}_{t+1|t} + K_{t+1}(\mathbf{z}_{t+1} - h(\hat{\mathbf{x}}_{t+1|t}))$$

$$P_{t+1|t+1} = (I - K_{t+1}H_{t+1})P_{t+1|t}$$

Range-Azimuth Observation

- We obtain i observations in ray space, i.e. a distance range and an azimuth angle: $z_t^i = (r_t^i, \phi_t^i)^\top$.
- The location in cartesian coordinates can be expressed by:

$$\begin{pmatrix} \hat{\mu}_{j,x} \\ \hat{\mu}_{j,y} \end{pmatrix} = \begin{pmatrix} \hat{\mu}_{t,x} \\ \hat{\mu}_{t,y} \end{pmatrix} + \begin{pmatrix} r_t^i \cos(\phi_t^i + \hat{\mu}_{t,\theta}) \\ r_t^i \sin(\phi_t^i + \hat{\mu}_{t,\theta}) \end{pmatrix}.$$

location of landmark j — estimated robot location — relative measurement

Note: Using $\hat{\mathbf{x}} = (\mu_{r,x}, \mu_{r,y}, \theta r, \mu_{1,x}, \mu_{1,y}, \ldots, \mu_{N,x}, \mu_{N,y})$ to avoid variable name confusion.

- If the landmark was previously unobserved, we initialize it with the above equation.
- We can now compute the expected observation:

$$\begin{pmatrix} \delta_x \\ \delta_y \end{pmatrix} = \begin{pmatrix} \hat{\mu}_{j,x} - \hat{\mu}_{t,x} \\ \hat{\mu}_{j,y} - \hat{\mu}_{t,y} \end{pmatrix}$$

$$\hat{\mathbf{z}}_t^i = h(\hat{\mu}_t) = \begin{pmatrix} \sqrt{\delta_x^2 + \delta_y^2} \\ \arctan \frac{\delta_y}{\delta_x} - \hat{\mu}_{t,\theta} \end{pmatrix}$$

Jacobian for the Observation

- Compute the Jacobian for $\hat{\mathbf{z}}_t^i = h(\hat{\mu}_t) = \begin{pmatrix} \sqrt{\delta_x^2 + \delta_y^2} \\ \arctan \frac{\delta_y}{\delta_x} - \hat{\mu}_{t,\theta} \end{pmatrix}$.
- This is again not touching the entire state vector, but only the motion state and the j-th landmark:

$$(x, y, \theta, m_{j,x}, m_{j_y})^\top$$

- Therefore we only need to compute a low-dimensional "excerpt" of H: $H_{t,\text{low}}^i = \frac{\partial h(\hat{\mu}_t)}{\partial \mu_t}$
- For brevity, we introduce $q = \delta_x^2 + \delta_y^2$.

$$\frac{\partial h_r}{\partial x} = \frac{\partial \sqrt{q}}{\partial x} = -\frac{1}{2\sqrt{q}} \cdot 2\delta_x = -\frac{\delta_x}{\sqrt{q}}, \quad \frac{\partial h_r}{\partial y} = -\frac{\delta_y}{\sqrt{q}}, \quad \frac{\partial h_r}{\partial \theta} = 0, \quad \frac{\partial h_r}{\partial m_{j,x}} = \frac{\delta_x}{\sqrt{q}}, \quad \frac{\partial h_r}{\partial m_{j,y}} = \frac{\delta_y}{\sqrt{q}}$$

$$\frac{\partial h_\theta}{\partial x} = \frac{\partial \arctan \frac{\delta_y}{\delta_x}}{\partial x} = \frac{1}{1 + (\frac{\delta_y}{\delta_x})^2} \cdot \frac{\delta_y}{\delta_x^2} = \frac{\delta_x^2}{\delta_x^2 + \delta_y^2} \cdot \frac{\delta_y}{\delta_x^2} = \frac{\delta_y}{q},$$

$$\frac{\partial h_\theta}{\partial y} = -\frac{\delta_x}{q}, \quad \frac{\partial h_\theta}{\partial \theta} = -1, \quad \frac{\partial h_\theta}{\partial m_{j,x}} = -\frac{\delta_y}{q}, \quad \frac{\partial h_\theta}{\partial m_{j,y}} = \frac{\delta_x}{q}.$$

Jacobian for the Observation

- With the above, we can write

$$H^i_{t,\text{low}} = \frac{1}{q}\begin{pmatrix} -\sqrt{q}\delta_x & -\sqrt{q}\delta_y & 0 & \sqrt{q}\delta_x & \sqrt{q}\delta_y \\ \delta_y & -\delta_x & -q & -\delta_y & \delta_x \end{pmatrix}.$$

- We can transfer it to high-dimensional space by multiplying with a mapping matrix $M_{x,j}$:

$$M_{x,j} = \begin{pmatrix} 1 & 0 & 0 & 0\ldots0 & 0 & 0 & 0\ldots0 \\ 0 & 1 & 0 & 0\ldots0 & 0 & 0 & 0\ldots0 \\ 0 & 0 & 1 & 0\ldots0 & 0 & 0 & 0\ldots0 \\ 0 & 0 & 1 & 0\ldots0 & 1 & 0 & 0\ldots0 \\ 0 & 0 & 1 & \underbrace{0\ldots0}_{2j-2} & 0 & 1 & \underbrace{0\ldots0}_{2N-2j} \end{pmatrix}$$

$$H^i_t = H^i_{t,\text{low}} M_{x,j}.$$

$$\hat{\mathbf{x}}_{t+1|t} = f(\hat{\mathbf{x}}_{t|t}, \mathbf{u}_t)$$

$$P_{t+1|t} = FP_{t|t}F^\top + Q_t$$

$$K_{t+1} = P_{t+1|t}H^\top_{t+1}(H_{t+1}P_{t+1|t}H^\top_{t+1} + R_{t+1})^{-1}$$

$$\hat{\mathbf{x}}_{t+1|t+1} = \hat{\mathbf{x}}_{t+1|t} + K_{t+1}(\mathbf{z}_{t+1} - h(\hat{\mathbf{x}}_{t+1|t}))$$

$$P_{t+1|t+1} = (I - K_{t+1}H_{t+1})P_{t+1|t}$$

Full Correction Step

1: **for** all observed features $z_t^i = (r_t^i, \phi_t^i)$ **do**

2: $\quad j = c_t^i$

3: $\quad$ **if** landmark j never seen before **then**

4: $\quad\quad \begin{pmatrix} \hat{\mu}_{j,x} \\ \hat{\mu}_{j,y} \end{pmatrix} = \begin{pmatrix} \hat{\mu}_{t,x} \\ \hat{\mu}_{t,y} \end{pmatrix} + \begin{pmatrix} r_t^i \cos(\phi_t^i + \hat{\mu}_{t,\theta} \\ r_t^i \sin(\phi_t^i + \hat{\mu}_{t,\theta} \end{pmatrix}$

5: $\quad$ **end if**

6: $\quad \begin{pmatrix} \delta_x \\ \delta_y \end{pmatrix} = \begin{pmatrix} \hat{\mu}_{j,x} - \hat{\mu}_{t,x} \\ \hat{\mu}_{j,y} - \hat{\mu}t, y \end{pmatrix}$

7: $\quad q = \delta_x^2 + \delta_y^2$

8: $\quad \hat{\mathbf{z}}_t^i = \begin{pmatrix} \sqrt{q} \\ \arctan \frac{\delta_y}{\delta_x} - \hat{\mu}_{t,\theta} \end{pmatrix}$

9: $\quad M_{x,j} = \ldots$

10: $\quad H_t^i = \frac{1}{q} \begin{pmatrix} -\sqrt{q}\delta_x & -\sqrt{q}\delta_y & 0 & \sqrt{q}\delta_x & \sqrt{q}\delta_y \\ \delta_y & -\delta_x & -q & -\delta_y & \delta_x \end{pmatrix} M_{x,j}$

11: $\quad$ Update K, $\hat{\mu}_t$, Σ_t

12: **end for**

$$\hat{\mathbf{x}}_{t+1|t} = f(\hat{\mathbf{x}}_{t|t}, \mathbf{u}_t)$$

$$P_{t+1|t} = FP_{t|t}F^\top + Q_t$$

$$K_{t+1} = P_{t+1|t}H_{t+1}^\top (H_{t+1}P_{t+1|t}H_{t+1}^\top + R_{t+1})^{-1}$$

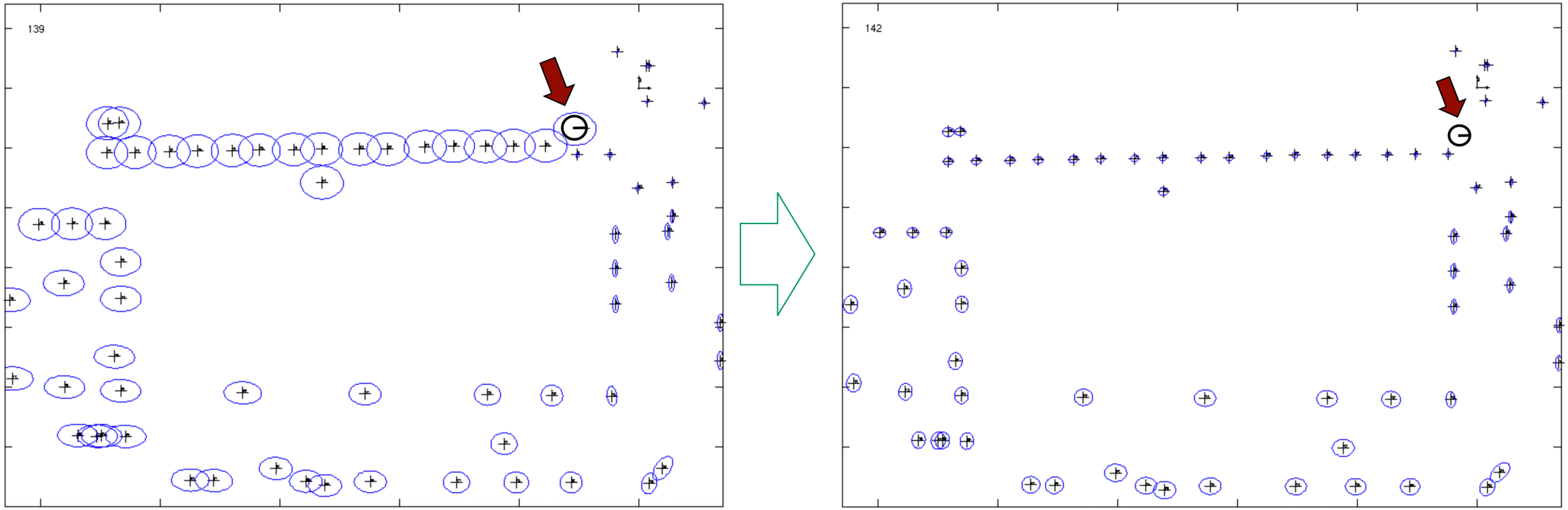
$$\hat{\mathbf{x}}_{t+1|t+1} = \hat{\mathbf{x}}_{t+1|t} + K_{t+1}(\mathbf{z}_{t+1} - h(\hat{\mathbf{x}}_{t+1|t}))$$

$$P_{t+1|t+1} = (I - K_{t+1}H_{t+1})P_{t+1|t}$$

Done!

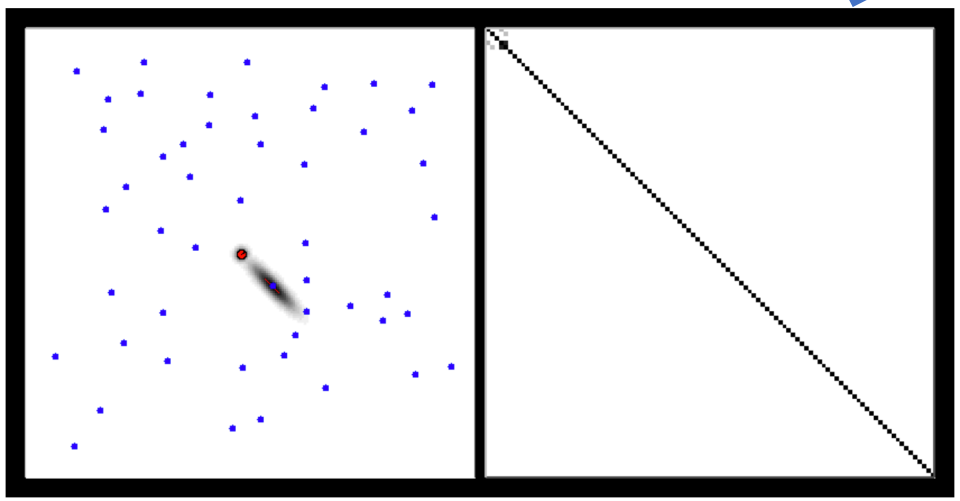
Loop Closing and Correlations

Loop Closing

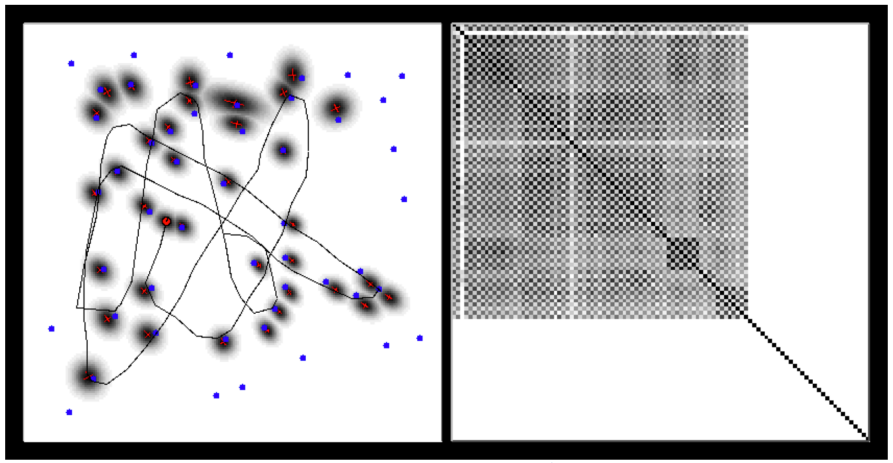


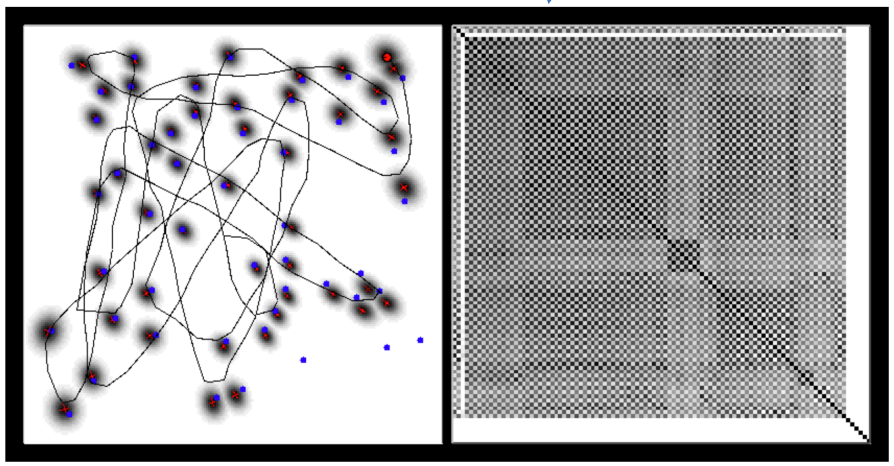
EKF-SLAM Correlations

- Correlations can be observed by the state of the covariance matrix P:



Map Correlation Matrix





EKF-SLAM: Complexity

- Cubic complexity w.r.t. the measurement dimensionality
- Cost per step: dominated by the number of landmarks: $O(n^2)$
- Memory consumption: $O(n^2)$
- The EKF becomes computationally impractical or even intractable for large maps!

Fast-SLAM: SLAM with Particle Filters

Particle Representation

- A set of weighted samples $\mathcal{X} = \{(\mathbf{x}^i, w^i)\}_{i=1,\dots,N}$
- Think of a sample as one hypothesis about the state.
- For feature-based SLAM: $\mathbf{x} = (\underbrace{x_{1:t}, y_{1:t}, \theta_{1:t}}_{\text{pose}}, \underbrace{l_{1,x}, l_{1,y}, \dots, l_{M,x}, l_{M,y}}_{\text{landmarks}})^\top$

Particle Representation

- **A set of weighted samples** $\mathcal{X} = \{(\mathbf{x}^i, w^i)\}_{i=1,\dots,N}$
- **Think of a sample as one hypothesis about the state.**
- **For feature-based SLAM:** $\mathbf{x} = (\underbrace{x_{1:t}, y_{1:t}, \theta_{1:t}}_{\text{pose}}, \underbrace{l_{1,x}, l_{1,y}, \dots, l_{M,x}, l_{M,y}}_{\text{landmarks}})^\top$

Not really stochastically dependent!

- **The problem with this is the dimensionality.**
  - Particle filters need reasonably dense coverage of the state space with samples.
  - Therefore they are effective in low-dimensional spaces.
  - Covering the full state space of $\mathbf{x} = (x_{1:t}, y_{1:t}, \theta_{1:t}, l_{1,x}, l_{1,y}, \dots, l_{M,x}, l_{M,y})^\top$ densely with samples is just too expensive.
  - It also does not deal well with map management (adding landmarks as they emerge, removing them as they become invalid or irrelevant) since that would require touching every sample every time.
- **How can we simplify the problem?**

Can we exploit or decouple dependencies?

- **If we know the poses of the robot, mapping is easy.**

$$\mathbf{x} = (x_{1:t}, y_{1:t}, \theta_{1:t}, l_{1,x}, l_{1,y}, \ldots, l_{M,x}, l_{M,y})^\top$$

- **Key idea:**
  - Use the particle set only to model the robot's path.
  - Each sample is a path hypothesis.
  - For each sample, we can then compute an individual map of landmarks.
  - The map depends on the poses of the robot.
  - We know how to build a map if the position of the sensor is known.
  - Given known robot pose, we can estimate locations of all features independent of each other!

Factored Posterior Landmarks

- **Notation: Instead of writing** $(x_{1:t}, y_{1:t}, \theta_{1:t}, l_{1,x}, l_{1,y}, \ldots, l_{M,x}, l_{M,y})^\top$,

  we will use the shorter $(\mathbf{x}_{1:t}, \mathbf{m}_{1:M})^\top$ **for the robot pose and the map.**

- **Bayesian representation:**

poses map observations movements

$$\underbrace{p(\mathbf{x}_{1:t}, \mathbf{m}_{1:M} | \mathbf{z}_{1:t}, \mathbf{u}_{1:t})}_{\text{SLAM posterior}} = \underbrace{p(\mathbf{x}_{1:t} | \mathbf{z}_{1:t}, \mathbf{u}_{1:t})}_{\text{Robot path posterior}} \underbrace{p(\mathbf{m}_{1:M} | \mathbf{x}_{1:t}, \mathbf{z}_{1:t})}_{\text{Map posterior}}$$

- **Does knowing the robot path help to solve the problem?**

Factored Posterior

- Using the assumption that all map features are independent, we can factorize the posterior:

$$p(\mathbf{x}_{1:t}, \mathbf{m}_{1:M} | \mathbf{z}_{1:t}, \mathbf{u}_{1:t}) = p(\mathbf{x}_{1:t} | \mathbf{z}_{1:t}, \mathbf{u}_{1:t}) p(\mathbf{m}_{1:M} | \mathbf{x}_{1:t}, \mathbf{z}_{1:t})$$

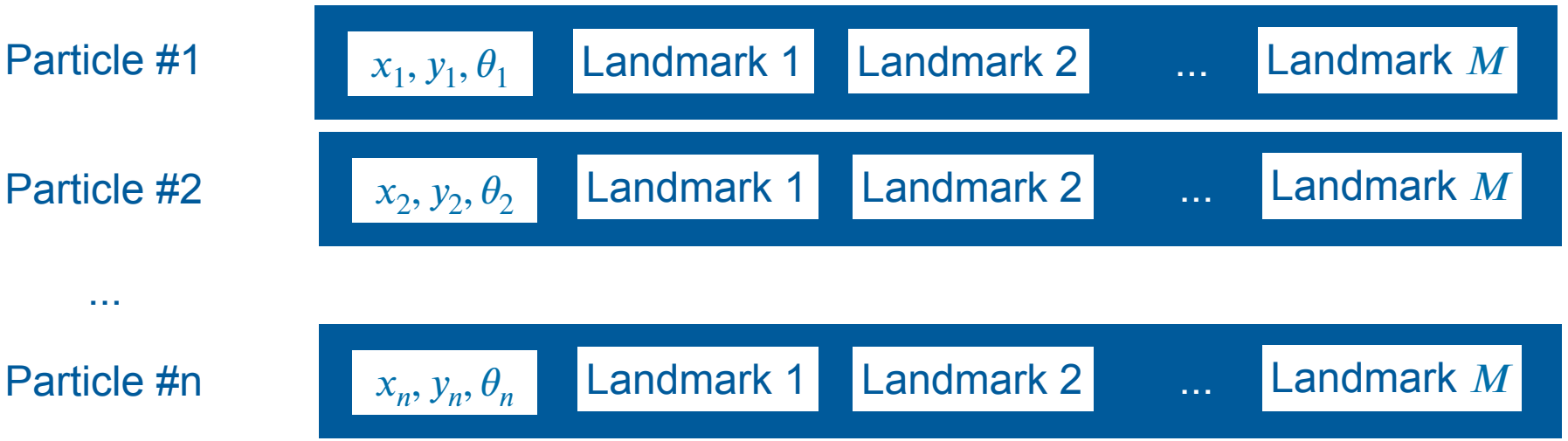
$$= \underbrace{p(\mathbf{x}_{1:t} | \mathbf{z}_{1:t}, \mathbf{u}_{1:t})}_{\text{Robot path posterior}} \underbrace{\prod_{i=1}^{M} p(l_i | \mathbf{x}_{1:t}, \mathbf{z}_{1:t})}_{\text{Landmark positions}}$$

Localization Problem — Mapping Problem

- This factorization is called *Rao-Blackwellization.* (Explanation and derivation in [1].)

Rao-Blackwellized Particle Filter – The FastSLAM Algorithm

- **FastSLAM [2] is a SLAM algorithm using Rao-Blackwellized Particle Filter / Kalman Filter combination.**
  1. estimate the robot pose using a particle filter (solving the localization problem),
  2. associate each particle with a map represented by a set of Gaussian-distributed landmark positions (solving the mapping problem),
  3. estimate the landmark positions using separate EKFs,
  4. compute particle weights based on correspondence between landmarks and observations (e.g. with Mahalanobis).



FastSLAM Key Steps

1. **Extend the past posterior by sampling a new pose for each sample:**

$$\mathbf{x}_t^{[k]} \sim p(\mathbf{x}_t | \mathbf{x}_{t-1}^{[k]}, \mathbf{u}_t)$$

2. **Compute particle weight based on observation weighted by covariance (Mahalanobis Distance):**

$$w^{[k]} = \frac{1}{\sqrt{2\pi Q}} e^{-\frac{(\mathbf{z}_t - \hat{\mathbf{z}}^{[k]})^\top (\mathbf{z}_t - \hat{\mathbf{z}}^{[k]})}{2Q}}$$

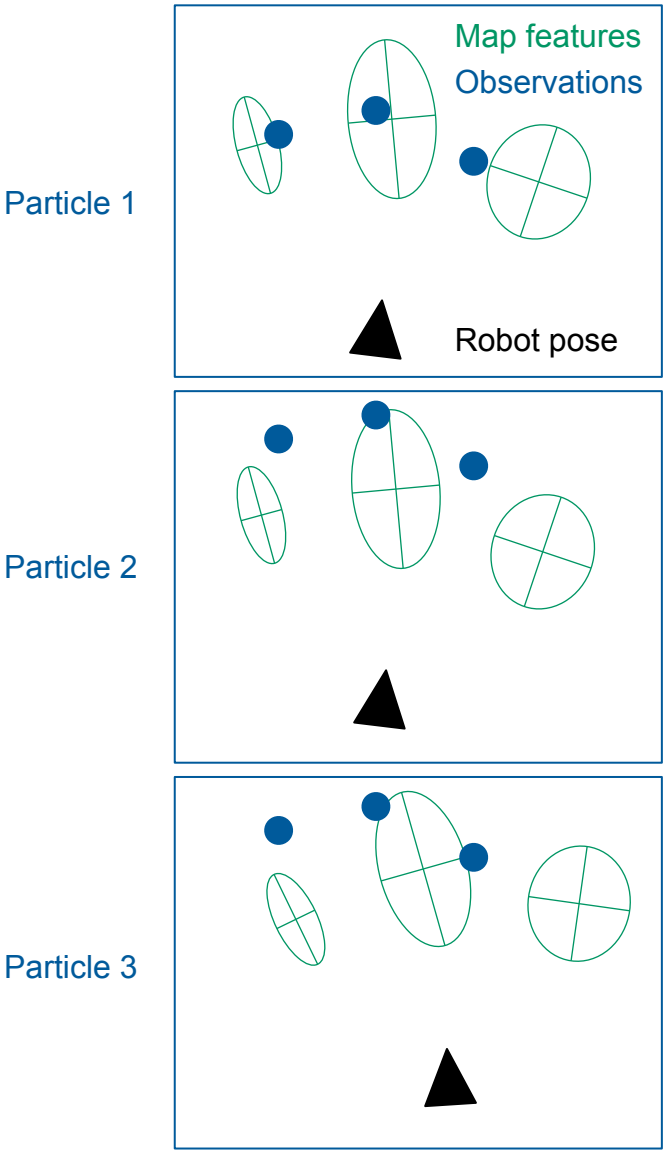
Here,

1. $Q = Q_t + H\Sigma_{j,t-1}^{[k]} H^\top$ is the measurement covariance obtained from measurement noise and landmark uncertainty (derivation: See appendix),
2. $\mathbf{z}_t - \hat{\mathbf{z}}^{[k]}$ represents the difference between the vector $\hat{\mathbf{z}}^{[k]}$ of all features observed at time t, and the vector $\mathbf{z}_t$ of their corresponding map features in particle k.

3. **Update belief of observed landmarks using the EKF formula.**
4. **Resample according to particle weights $w^{[k]}$.**

FastSLAM 1.0 → 2.0

- **FastSLAM 2.0 samples from** $x_t^{[k]} \sim p(x_t | x_{1:t-1}^{[k]}, u_{1:t}, z_{1:t}).$
- **I.e., it takes into account the sensor data.**
- **For example,**
  - multiply weights with average Mahalanobis distance of map features to observations (*matching criterion)*
  - use least-squares method to optimize position parameters based on map/observation matches, then weigh based on some distance measure of the particle to that "optimal" position
  - ...
- **This results in a more peaked proposal distribution.**
- **Less particles required, more robust and accurate**
- **But more complex...**



FastSLAM Summary

- **Particle filter-based SLAM**
- **Rao-Blackwellization: Model the robot's path by sampling, and compute the landmarks given the poses**
- **Allows for simple per-particle association, inherently addressing multi-modality and ambiguities**
- **FastSLAM 1.0 and 2.0 differ in the proposal distribution**
- **Complexity $O(NM)$ with naive data structures, $O\left(N \log M\right)$ with smarter setup**
- **Scales well (>1,000,000 features no problem)**
- **Much stronger than classical EKF approach.**



Technische Hochschule Ingolstadt

Fakultät Elektro- und Informationstechnik

Thank you very much!